

Chapter 11

$\mathcal{NP}$ -HARD AND $\mathcal{NP}$ -COMPLETE PROBLEMS

11.1 BASIC CONCEPTS

In this chapter we are concerned with the distinction between problems that can be solved by a polynomial time algorithm and problems for which no polynomial time algorithm is known. It is an unexplained phenomenon that for many of the problems we know and study, the best algorithms for their solutions have computing times that cluster into two groups. The first group consists of problems whose solution times are bounded by polynomials of small degree. Examples we have seen in this book include ordered searching, which is $O(\log n)$, polynomial evaluation which is $O(n)$, sorting which is $O(n \log n)$, and string editing which is $O(mn)$.

The second group is made up of problems whose best-known algorithms are nonpolynomial. Examples we have seen include the traveling salesperson and the knapsack problems for which the best algorithms given in this text have complexities $O(n^2 2^n)$ and $O(2^{n/2})$ respectively. In the quest to develop efficient algorithms, no one has been able to develop a polynomial time algorithm for any problem in the second group. This is very important because algorithms whose computing times are greater than polynomial (typically the time is exponential) very quickly require such vast amounts of time to execute that even moderate-size problems cannot be solved (see Section 1.3 for more details).

The theory of $\mathcal{NP}$ -completeness which we present here does not provide a method of obtaining polynomial time algorithms for problems in the second group. Nor does it say that algorithms of this complexity do not exist. Instead, what we do is show that many of the problems for which there are

no known polynomial time algorithms are computationally related. In fact, we establish two classes of problems. These are given the names $\mathcal{NP}$ -hard and $\mathcal{NP}$ -complete. A problem that is $\mathcal{NP}$ -complete has the property that it can be solved in polynomial time if and only if all other $\mathcal{NP}$ -complete problems can also be solved in polynomial time. If an $\mathcal{NP}$ -hard problem can be solved in polynomial time, then all $\mathcal{NP}$ -complete problems can be solved in polynomial time. All $\mathcal{NP}$ -complete problems are $\mathcal{NP}$ -hard, but some $\mathcal{NP}$ -hard problems are not known to be $\mathcal{NP}$ -complete.

Although one can define many distinct problem classes having the properties stated above for the $\mathcal{NP}$ -hard and $\mathcal{NP}$ -complete classes, the classes we study are related to nondeterministic computations (to be defined later). The relationship of these classes to nondeterministic computations together with the apparent power of nondeterminism leads to the intuitive (though as yet unproved) conclusion that no $\mathcal{NP}$ -complete or $\mathcal{NP}$ -hard problem is polynomially solvable.

We see that the class of $\mathcal{NP}$ -hard problems (and the subclass of $\mathcal{NP}$ -complete problems) is very rich as it contains many interesting problems from a wide variety of disciplines. First, we formalize the preceding discussion of the classes.

11.1.1 Nondeterministic Algorithms

Up to now the notion of algorithm that we have been using has the property that the result of every operation is uniquely defined. Algorithms with this property are termed *deterministic algorithms*. Such algorithms agree with the way programs are executed on a computer. In a theoretical framework we can remove this restriction on the outcome of every operation. We can allow algorithms to contain operations whose outcomes are not uniquely defined but are limited to specified sets of possibilities. The machine executing such operations is allowed to choose any one of these outcomes subject to a termination condition to be defined later. This leads to the concept of a *nondeterministic algorithm*. To specify such algorithms, we introduce three new functions:

1. `Choice( $S$ )` arbitrarily chooses one of the elements of set S .
2. `Failure()` signals an unsuccessful completion.
3. `Success()` signals a successful completion.

The assignment statement $x := \text{Choice}(1, n)$ could result in x being assigned any one of the integers in the range $[1, n]$. There is no rule specifying how this choice is to be made. The `Failure()` and `Success()` signals are used to define a computation of the algorithm. These statements cannot be used to effect a **return**. Whenever there is a set of choices that leads to a successful

completion, then one such set of choices is always made and the algorithm terminates successfully. A *nondeterministic algorithm terminates unsuccessfully if and only if there exists no set of choices leading to a success signal*. The computing times for Choice, Success, and Failure are taken to be $O(1)$. A machine capable of executing a nondeterministic algorithm in this way is called a *nondeterministic machine*. Although nondeterministic machines (as defined here) do not exist in practice, we see that they provide strong intuitive reasons to conclude that certain problems cannot be solved by fast deterministic algorithms.

Example 11.1 Consider the problem of searching for an element x in a given set of elements $A[1 : n]$, $n \geq 1$. We are required to determine an index j such that $A[j] = x$ or $j = 0$ if x is not in A . A nondeterministic algorithm for this is Algorithm 11.1.

```

1   $j := \text{Choice}(1, n);$ 
2  if  $A[j] = x$  then {write  $(j)$ ; Success();}
3  write  $(0)$ ; Failure();
```

Algorithm 11.1 Nondeterministic search

From the way a nondeterministic computation is defined, it follows that the number 0 can be output if and only if there is no j such that $A[j] = x$. Algorithm 11.1 is of nondeterministic complexity $O(1)$. Note that since A is not ordered, every deterministic search algorithm is of complexity $\Omega(n)$. $\square$

Example 11.2 [Sorting] Let $A[i]$, $1 \leq i \leq n$, be an unsorted array of positive integers. The nondeterministic algorithm $\text{NSort}(A, n)$ (Algorithm 11.2) sorts the numbers into nondecreasing order and then outputs them in this order. An auxiliary array $B[1 : n]$ is used for convenience. Line 4 initializes B to zero though any value different from all the $A[i]$ will do. In the **for** loop of lines 5 to 10, each $A[i]$ is assigned to a position in B . Line 7 nondeterministically determines this position. Line 8 ascertains that $B[j]$ has not already been used. Thus, the order of the numbers in B is some permutation of the initial order in A . The **for** loop of lines 11 and 12 verifies that B is sorted in nondecreasing order. A successful completion is achieved if and only if the numbers are output in nondecreasing order. Since there is always a set of choices at line 7 for such an output order, algorithm NSort is a sorting algorithm. Its complexity is $O(n)$. Recall that all deterministic sorting algorithms must have a complexity $\Omega(n \log n)$. $\square$

```

1  Algorithm NSort( $A, n$ )
2  // Sort  $n$  positive integers.
3  {
4      for  $i := 1$  to  $n$  do  $B[i] := 0$ ; // Initialize  $B[ ]$ .
5      for  $i := 1$  to  $n$  do
6          {
7               $j := \text{Choice}(1, n)$ ;
8              if  $B[j] \neq 0$  then Failure();
9               $B[j] := A[i]$ ;
10         }
11     for  $i := 1$  to  $n - 1$  do // Verify order.
12         if  $B[i] > B[i + 1]$  then Failure();
13     write ( $B[1 : n]$ );
14     Success();
15 }
```

Algorithm 11.2 Nondeterministic sorting

A deterministic interpretation of a nondeterministic algorithm can be made by allowing unbounded parallelism in computation. In theory, each time a choice is to be made, the algorithm makes several copies of itself. One copy is made for each of the possible choices. Thus, many copies are executing at the same time. The first copy to reach a successful completion terminates all other computations. If a copy reaches a failure completion, then only that copy of the algorithm terminates. Although this interpretation may enable one to better understand nondeterministic algorithms, it is important to remember that a nondeterministic machine does not make any copies of an algorithm every time a choice is to be made. Instead, it has the ability to select a “correct” element from the set of allowable choices (if such an element exists) every time a choice is to be made. A correct element is defined relative to a shortest sequence of choices that leads to a successful termination. In case there is no sequence of choices leading to a successful termination, we assume that the algorithm terminates in one unit of time with output “unsuccessful computation.” Whenever successful termination is possible, a nondeterministic machine makes a sequence of choices that is a shortest sequence leading to a successful termination. Since, the machine we are defining is fictitious, it is not necessary for us to concern ourselves with how the machine can make a correct choice at each step.

Definition 11.1 Any problem for which the answer is either zero or one is called a *decision problem*. An algorithm for a decision problem is termed

a *decision algorithm*. Any problem that involves the identification of an optimal (either minimum or maximum) value of a given cost function is known as an *optimization problem*. An *optimization algorithm* is used to solve an optimization problem. $\square$

It is possible to construct nondeterministic algorithms for which many different choice sequences lead to successful completions. Algorithm NSort of Example 11.2 is one such algorithm. If the numbers $A[i]$ are not distinct, then many different permutations will result in a sorted sequence. If NSort were written to output the permutation used rather than the $A[i]$'s in sorted order, then its output would not be uniquely defined. We concern ourselves only with those nondeterministic algorithms that generate unique outputs. In particular we consider only nondeterministic decision algorithms. A successful completion is made if and only if the output is 1. A 0 is output if and only if there is no sequence of choices leading to a successful completion. The output statement is implicit in the signals Success and Failure. No explicit output statements are permitted in a decision algorithm. Clearly, our earlier definition of a nondeterministic computation implies that the output from a decision algorithm is uniquely defined by the input parameters and algorithm specification.

Although the idea of a decision algorithm may appear very restrictive at this time, many optimization problems can be recast into decision problems with the property that the decision problem can be solved in polynomial time if and only if the corresponding optimization problem can. In other cases, we can at least make the statement that if the decision problem cannot be solved in polynomial time, then the optimization problem cannot either.

Example 11.3 [Maximum clique] A maximal complete subgraph of a graph $G = (V, E)$ is a *clique*. The size of the clique is the number of vertices in it. The *max clique problem* is an optimization problem that has to determine the size of a largest clique in G . The corresponding decision problem is to determine whether G has a clique of size at least k for some given k . Let $\text{DClique}(G, k)$ be a deterministic decision algorithm for the clique decision problem. If the number of vertices in G is n , the size of a max clique in G can be found by making several applications of DClique. DClique is used once for each k , $k = n, n-1, n-2, \dots$, until the output from DClique is 1. If the time complexity of DClique is $f(n)$, then the size of a max clique can be found in time $\leq n f(n)$. Also, if the size of a max clique can be determined in time $g(n)$, then the decision problem can be solved in time $g(n)$. Hence, the max clique problem can be solved in polynomial time if and only if the clique decision problem can be solved in polynomial time. $\square$

Example 11.4 [0/1 knapsack] The knapsack decision problem is to determine whether there is a 0/1 assignment of values to x_i , $1 \leq i \leq n$, such that $\sum p_i x_i \geq r$ and $\sum w_i x_i \leq m$. The r is a given number. The p_i 's and w_i 's are

nonnegative numbers. If the knapsack decision problem cannot be solved in deterministic polynomial time, then the optimization problem cannot either. $\square$

Before proceeding further, it is necessary to arrive at a uniform parameter n to measure complexity. We assume that n is the length of the input to the algorithm (that is, n is the input size). We also assume that all inputs are integer. Rational inputs can be provided by specifying pairs of integers. Generally, the length of an input is measured assuming a binary representation; that is, if the number 10 is to be input, then in binary it is represented as 1010. Its length is 4. In general, a positive integer k has a length of $\lfloor \log_2 k \rfloor + 1$ bits when represented in binary. The length of the binary representation of 0 is 1. The size, or length, n of the input to an algorithm is the sum of the lengths of the individual numbers being input. In case the input is given using a different representation (say radix r), then the length of a positive number k is $\lfloor \log_r k \rfloor + 1$. Thus, in decimal notation, $r = 10$ and the number 100 has a length $\log_{10} 100 + 1 = 3$. Since $\log_r k = \log_2 k / \log_2 r$, the length of any input using radix r ($r > 1$) representation is $c(r)n$, where n is the length using a binary representation and $c(r)$ is a number that is fixed for a given r .

When inputs are given using the radix $r = 1$, we say the input is in *unary form*. In unary form, the number 5 is input as 11111. Thus, the length of a positive integer k is k . It is important to observe that the length of a unary input is exponentially related to the length of the corresponding r -ary input for radix r , $r > 1$.

Example 11.5 [Max clique] The input to the max clique decision problem can be provided as a sequence of edges and an integer k . Each edge in $E(G)$ is a pair of numbers (i, j) . The size of the input for each edge (i, j) is $\lfloor \log_2 i \rfloor + \lfloor \log_2 j \rfloor + 2$ if a binary representation is assumed. The input size of any instance is

$$n = \sum_{\substack{(i,j) \in E(G) \\ i < j}} (\lfloor \log_2 i \rfloor + \lfloor \log_2 j \rfloor + 2) + \lfloor \log_2 k \rfloor + 1$$

Note that if G has only one connected component, then $n \geq |V|$. Thus, if this decision problem cannot be solved by an algorithm of complexity $p(n)$ for some polynomial $p()$, then it cannot be solved by an algorithm of complexity $p(|V|)$. $\square$

Example 11.6 [0/1 knapsack] Assuming p_i, w_i, m , and r are all integers, the input size for the knapsack decision problem is

$$q = \sum_{1 \leq i \leq n} (\lfloor \log_2 p_i \rfloor + \lfloor \log_2 w_i \rfloor) + 2n + \lfloor \log_2 m \rfloor + \lfloor \log_2 r \rfloor + 2$$

Note that $q > n$. If the input is given in unary notation, then the input size s is $\sum p_i + \sum w_i + m + r$. Note that the knapsack decision and optimization problems can be solved in time $p(s)$ for some polynomial $p()$ (see the dynamic programming algorithm). However, there is no known algorithm with complexity $O(p(n))$ for some polynomial $p()$. $\square$

We are now ready to formally define the complexity of a nondeterministic algorithm.

Definition 11.2 The *time required by a nondeterministic algorithm* performing on any given input is the minimum number of steps needed to reach a successful completion if there exists a sequence of choices leading to such a completion. In case successful completion is not possible, then the time required is $O(1)$. A nondeterministic algorithm is of complexity $O(f(n))$ if for all inputs of size n , $n \geq n_0$, that result in a successful completion, the time required is at most $cf(n)$ for some constants c and n_0 . $\square$

In Definition 11.2 we assume that each computation step is of a fixed cost. In word-oriented computers this is guaranteed by the finiteness of each word. When each step is not of a fixed cost, it is necessary to consider the cost of individual instructions. Thus, the addition of two m -bit numbers takes $O(m)$ time, their multiplication takes $O(m^2)$ time (using classical multiplication), and so on. To see the necessity of this, consider the algorithm Sum (Algorithm 11.3). This is a deterministic algorithm for the sum of subsets decision problem. It uses an $(m+1)$ -bit word s . The i th bit in s is zero if and only if no subset of the integers $A[j]$, $1 \leq j \leq n$, sums to i . Bit 0 of s is always 1 and the bits are numbered $0, 1, 2, \dots, m$ right to left. The function Shift shifts the bits in s to the left by $A[i]$ bits. The total number of steps for this algorithm is only $O(n)$. However, each step moves $m+1$ bits of data and would take $O(m)$ time on a conventional computer. Assuming one unit of time is needed for each basic operation for a fixed word size, the complexity is $O(nm)$ and not $O(n)$.

The virtue of conceiving of nondeterministic algorithms is that often what would be very complex to write deterministically is very easy to write nondeterministically. In fact, it is very easy to obtain polynomial time nondeterministic algorithms for many problems that can be deterministically solved by a systematic search of a solution space of exponential size.

Example 11.7 [Knapsack decision problem] DKP (Algorithm 11.4) is a nondeterministic polynomial time algorithm for the knapsack decision problem. The **for** loop of lines 4 to 8 assigns 0/1 values to $x[i]$, $1 \leq i \leq n$. It also computes the total weight and profit corresponding to this choice of $x[\]$. Line 9 checks to see whether this assignment is feasible and whether the resulting profit is at least r . A successful termination is possible iff the answer to the decision problem is yes. The time complexity is $O(n)$. If q is the input length using a binary representation, the time is $O(q)$. $\square$

```

1  Algorithm Sum( $A$ ,  $n$ ,  $m$ )
2  {
3       $s := 1$ ;
4      //  $s$  is an  $(m + 1)$ -bit word. Bit zero is 1.
5      for  $i := 1$  to  $n$  do
6           $s := s$  or Shift( $s$ ,  $A[i]$ );
7          if the  $m$ th bit in  $s$  is 1 then
8              write ("A subset sums to  $m$ .");
9          else write ("No subset sums to  $m$ .");
10 }

```

Algorithm 11.3 Deterministic sum of subsets

Example 11.8 [Max clique] Algorithm DCK (Algorithm 11.5) is a nondeterministic algorithm for the clique decision problem. The algorithm begins by trying to form a set of k distinct vertices. Then it tests to see whether these vertices form a complete subgraph. If G is given by its adjacency matrix and $|V| = n$, the input length m is $n^2 + \lfloor \log_2 k \rfloor + \lfloor \log_2 n \rfloor + 2$. The **for** loop of lines 4 to 9 can easily be implemented to run in nondeterministic time $O(n)$. The time for the **for** loop of lines 11 and 12 is $O(k^2)$. Hence the overall nondeterministic time is $O(n + k^2) = O(n^2) = O(m)$. There is no known polynomial time deterministic algorithm for this problem. $\square$

Example 11.9 [Satisfiability] Let $x_1, x_2, \dots$ denote boolean variables (their value is either true or false). Let $\bar{x}_i$ denote the negation of x_i . A *literal* is either a variable or its negation. A formula in the propositional calculus is an expression that can be constructed using literals and the operations **and** and **or**. Examples of such formulas are $(x_1 \wedge x_2) \vee (x_3 \wedge \bar{x}_4)$ and $(x_3 \vee \bar{x}_4) \wedge (x_1 \vee \bar{x}_2)$. The symbol $\vee$ denotes **or** and $\wedge$ denotes **and**. A formula is in *conjunctive normal form* (CNF) if and only if it is represented as $\bigwedge_{i=1}^k c_i$, where the c_i are clauses each represented as $\bigvee l_{ij}$. The l_{ij} are literals. It is in *disjunctive normal form* (DNF) if and only if it is represented as $\bigvee_{i=1}^k c_i$ and each clause c_i is represented as $\bigwedge l_{ij}$. Thus $(x_1 \wedge x_2) \vee (x_3 \wedge \bar{x}_4)$ is in DNF whereas $(x_3 \vee \bar{x}_4) \wedge (x_1 \vee \bar{x}_2)$ is in CNF. The **satisfiability** problem is to determine whether a formula is true for some assignment of truth values to the variables. *CNF-satisfiability* is the satisfiability problem for CNF formulas.

It is easy to obtain a polynomial time nondeterministic algorithm that terminates successfully if and only if a given propositional formula $E(x_1, \dots, x_n)$ is satisfiable. Such an algorithm could proceed by simply choosing (nondeter-

```

1  Algorithm DKP( $p, w, n, m, r, x$ )
2  {
3       $W := 0; P := 0;$ 
4      for  $i := 1$  to  $n$  do
5          {
6               $x[i] := \text{Choice}(0, 1);$ 
7               $W := W + x[i] * w[i]; P := P + x[i] * p[i];$ 
8          }
9      if  $((W > m) \text{ or } (P < r))$  then Failure();
10     else Success();
11 }

```

Algorithm 11.4 Nondeterministic knapsack algorithm

```

1  Algorithm DCK( $G, n, k$ )
2  {
3       $S := \emptyset;$  //  $S$  is an initially empty set.
4      for  $i := 1$  to  $k$  do
5          {
6               $t := \text{Choice}(1, n);$ 
7              if  $t \in S$  then Failure();
8               $S := S \cup \{t\}$  // Add  $t$  to set  $S$ .
9          }
10     // At this point  $S$  contains  $k$  distinct vertex indices.
11     for all pairs  $(i, j)$  such that  $i \in S, j \in S$ , and  $i \neq j$  do
12         if  $(i, j)$  is not an edge of  $G$  then Failure();
13     Success();
14 }

```

Algorithm 11.5 Nondeterministic clique pseudocode

ministically) one of the 2^n possible assignments of truth values to $(x_1, \dots, x_n)$ and verifying that $E(x_1, \dots, x_n)$ is true for that assignment.

Eval (Algorithm 11.6) does this. The nondeterministic time required by the algorithm is $O(n)$ to choose the value of $(x_1, \dots, x_n)$ plus the time needed to deterministically evaluate E for that assignment. This time is proportional to the length of E . $\square$

```

1  Algorithm Eval( $E, n$ )
2  // Determine whether the propositional formula  $E$  is
3  // satisfiable. The variables are  $x_1, x_2, \dots, x_n$ .
4  {
5      for  $i := 1$  to  $n$  do // Choose a truth value assignment.
6           $x_i := \text{Choice}(\text{false}, \text{true});$ 
7      if  $E(x_1, \dots, x_n)$  then Success();
8      else Failure();
9  }
```

Algorithm 11.6 Nondeterministic satisfiability

11.1.2 The Classes $\mathcal{NP}$ -hard and $\mathcal{NP}$ -complete

In measuring the complexity of an algorithm, we use the input length as the parameter. An algorithm A is of *polynomial complexity* if there exists a polynomial $p()$ such that the computing time of A is $O(p(n))$ for every input of size n .

Definition 11.3 $\mathcal{P}$ is the set of all decision problems solvable by deterministic algorithms in polynomial time. $\mathcal{NP}$ is the set of all decision problems solvable by nondeterministic algorithms in polynomial time. $\square$

Since deterministic algorithms are just a special case of nondeterministic ones, we conclude that $\mathcal{P} \subseteq \mathcal{NP}$. What we do not know, and what has become perhaps the most famous unsolved problem in computer science, is whether $\mathcal{P} = \mathcal{NP}$ or $\mathcal{P} \neq \mathcal{NP}$.

Is it possible that for all the problems in $\mathcal{NP}$, there exist polynomial time deterministic algorithms that have remained undiscovered? This seems unlikely, at least because of the tremendous effort that has already been expended by so many people on these problems. Nevertheless, a proof that $\mathcal{P} \neq \mathcal{NP}$ is just as elusive and seems to require as yet undiscovered techniques.

But as with many famous unsolved problems, they serve to generate other useful results, and the question of whether $\mathcal{NP} \subseteq \mathcal{P}$ is no exception. Figure 11.1 displays the relationship between $\mathcal{P}$ and $\mathcal{NP}$ assuming that $\mathcal{P} \neq \mathcal{NP}$.

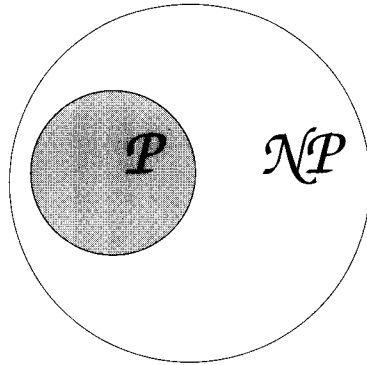


Figure 11.1 Commonly believed relationship between $\mathcal{P}$ and $\mathcal{NP}$

S. Cook formulated the following question: Is there any single problem in $\mathcal{NP}$ such that if we showed it to be in $\mathcal{P}$, then that would imply that $\mathcal{P} = \mathcal{NP}$? Cook answered his own question in the affirmative with the following theorem.

Theorem 11.1 [Cook] Satisfiability is in $\mathcal{P}$ if and only if $\mathcal{P} = \mathcal{NP}$.

Proof: See Section 11.2. □

We are now ready to define the $\mathcal{NP}$ -hard and $\mathcal{NP}$ -complete classes of problems. First we define the notion of reducibility. Note that this definition is similar to the one made in Section 10.3.

Definition 11.4 Let L_1 and L_2 be problems. Problem L_1 *reduces* to L_2 (also written $L_1 \propto L_2$) if and only if there is a way to solve L_1 by a deterministic polynomial time algorithm using a deterministic algorithm that solves L_2 in polynomial time. □

This definition implies that if we have a polynomial time algorithm for L_2 , then we can solve L_1 in polynomial time. One can readily verify that $\propto$ is a transitive relation (that is, if $L_1 \propto L_2$ and $L_2 \propto L_3$, then $L_1 \propto L_3$).

Definition 11.5 A problem L is $\mathcal{NP}$ -hard if and only if satisfiability reduces to L (satisfiability $\propto L$). A problem L is $\mathcal{NP}$ -complete if and only if L is $\mathcal{NP}$ -hard and $L \in \mathcal{NP}$. □

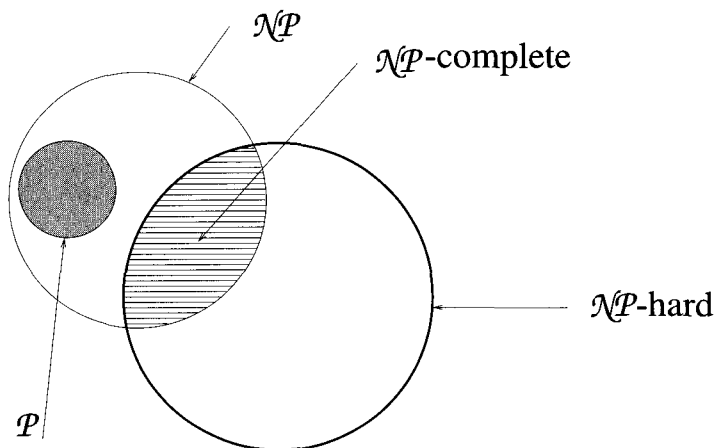


Figure 11.2 Commonly believed relationship among $\mathcal{P}$, $\mathcal{NP}$, $\mathcal{NP}$ -complete, and $\mathcal{NP}$ -hard problems

It is easy to see that there are $\mathcal{NP}$ -hard problems that are not $\mathcal{NP}$ -complete. Only a decision problem can be $\mathcal{NP}$ -complete. However, an optimization problem may be $\mathcal{NP}$ -hard. Furthermore if L_1 is a decision problem and L_2 an optimization problem, it is quite possible that $L_1 \propto L_2$. One can trivially show that the knapsack decision problem reduces to the knapsack optimization problem. For the clique problem one can easily show that the clique decision problem reduces to the clique optimization problem. In fact, one can also show that these optimization problems reduce to their corresponding decision problems (see the exercises). Yet, optimization problems cannot be $\mathcal{NP}$ -complete whereas decision problems can. There also exist $\mathcal{NP}$ -hard decision problems that are not $\mathcal{NP}$ -complete. Figure 11.2 shows the relationship among these classes.

Example 11.10 As an extreme example of an $\mathcal{NP}$ -hard decision problem that is not $\mathcal{NP}$ -complete consider the halting problem for deterministic algorithms. The *halting problem* is to determine for an arbitrary deterministic algorithm A and an input I whether algorithm A with input I ever terminates (or enters an infinite loop). It is well known that this problem is undecidable. Hence, there exists no algorithm (of any complexity) to solve this problem. So, it clearly cannot be in $\mathcal{NP}$. To show satisfiability $\propto$ the halting problem, simply construct an algorithm A whose input is a propositional formula X . If X has n variables, then A tries out all 2^n possible truth assignments and verifies whether X is satisfiable. If it is, then A stops. If it is not, then A enters an infinite loop. Hence, A halts on input X if and only

if X is satisfiable. If we had a polynomial time algorithm for the halting problem, then we could solve the satisfiability problem in polynomial time using A and X as input to the algorithm for the halting problem. Hence, the halting problem is an $\mathcal{NP}$ -hard problem that is not in $\mathcal{NP}$. $\square$

Definition 11.6 Two problems L_1 and L_2 are said to be *polynomially equivalent* if and only if $L_1 \propto L_2$ and $L_2 \propto L_1$. $\square$

To show that a problem L_2 is $\mathcal{NP}$ -hard, it is adequate to show $L_1 \propto L_2$, where L_1 is some problem already known to be $\mathcal{NP}$ -hard. Since $\propto$ is a transitive relation, it follows that if satisfiability $\propto L_1$ and $L_1 \propto L_2$, then satisfiability $\propto L_2$. To show that an $\mathcal{NP}$ -hard decision problem is $\mathcal{NP}$ -complete, we have just to exhibit a polynomial time nondeterministic algorithm for it.

Later sections show many problems to be $\mathcal{NP}$ -hard. Although we restrict ourselves to decision problems, it should be clear that the corresponding optimization problems are also $\mathcal{NP}$ -hard. The $\mathcal{NP}$ -completeness proofs are left as exercises (for those problems that are $\mathcal{NP}$ -complete).

EXERCISES

1. Given two sets S_1 and S_2 , the disjoint sets problem is to check whether the sets have a common element (see Section 10.3.2). Present an $O(1)$ time nondeterministic algorithm for this problem.
2. Given a sequence of n numbers, the distinct elements problem is to check if there are equal numbers (see Section 10.3, Exercise 5). Give an $O(1)$ time nondeterministic algorithm for this problem.
3. Obtain a nondeterministic algorithm of complexity $O(n)$ to determine whether there is a subset of n numbers a_i , $1 \leq i \leq n$, that sums to m .
4. (a) Show that the knapsack optimization problem reduces to the knapsack decision problem when all the p 's, w 's, and m are integer and the complexity is measured as a function of input length. (*Hint:* If the input length is q , then $\sum p_i \leq n2^q$, where n is the number of objects. Use a binary search to determine the optimal solution value.)
 (b) Let DK be an algorithm for the knapsack decision problem. Let r be the value of an optimal solution to the knapsack optimization problem. Show how to obtain a 0/1 assignment for the x_i , $1 \leq i \leq n$, such that $\sum p_i x_i = r$ and $\sum w_i x_i \leq m$ by making n applications of DK.

5. Show that the clique optimization problem reduces to the clique decision problem.
6. Let $\text{Sat}(E)$ be an algorithm to determine whether a propositional formula E in CNF is satisfiable. Show that if E is satisfiable and has n variables $x_1, x_2, \dots, x_n$, then using $\text{Sat}(E)$ n times, one can determine a truth value assignment for the x_i 's for which E is true.
7. Let π_2 be a problem for which there exists a deterministic algorithm that runs in time $2^{\sqrt{n}}$ (where n is the input size). Prove or disprove:

If π_1 is another problem such that π_1 is polynomially reducible to π_2 , then π_1 can be solved in deterministic $O(2^{\sqrt{n}})$ time on any input of size n .

11.2 COOK'S THEOREM (*)

Cook's theorem (Theorem 11.1) states that satisfiability is in $\mathcal{P}$ if and only if $\mathcal{P} = \mathcal{NP}$. We now prove this important theorem. We have already seen that satisfiability is in $\mathcal{NP}$ (Example 11.9). Hence, if $\mathcal{P} = \mathcal{NP}$, then satisfiability is in $\mathcal{P}$. It remains to be shown that if satisfiability is in $\mathcal{P}$, then $\mathcal{P} = \mathcal{NP}$. To do this, we show how to obtain from any polynomial time nondeterministic decision algorithm A and input I a formula $Q(A, I)$ such that Q is satisfiable iff A has a successful termination with input I . If the length of I is n and the time complexity of A is $p(n)$ for some polynomial $p()$, then the length of Q is $O(p^3(n) \log n) = O(p^4(n))$. The time needed to construct Q is also $O(p^3(n) \log n)$. A deterministic algorithm Z to determine the outcome of A on any input I can be easily obtained. Algorithm Z simply computes Q and then uses a deterministic algorithm for the satisfiability problem to determine whether Q is satisfiable. If $O(q(m))$ is the time needed to determine whether a formula of length m is satisfiable, then the complexity of Z is $O(p^3(n) \log n + q(p^3(n) \log n))$. If satisfiability is in $\mathcal{P}$, then $q(m)$ is a polynomial function of m and the complexity of Z becomes $O(r(n))$ for some polynomial $r()$. Hence, if satisfiability is in $\mathcal{P}$, then for every nondeterministic algorithm A in $\mathcal{NP}$ we can obtain a deterministic Z in $\mathcal{P}$. So, the above construction shows that if satisfiability is in $\mathcal{P}$, then $\mathcal{P} = \mathcal{NP}$.

Before going into the construction of Q from A and I , we make some simplifying assumptions on our nondeterministic machine model and on the form of A . These assumptions do not in any way alter the class of decision problems in $\mathcal{NP}$ or $\mathcal{P}$. The simplifying assumptions are as follows.

1. The machine on which A is to be executed is word oriented. Each word is w bits long. Multiplication, addition, subtraction, and so on,

between numbers one word long take one unit of time. If numbers are longer than a word, then the corresponding operations take at least as many units as the number of words making up the longest number.

2. A *simple expression* is an expression that contains at most one operator and all operands are simple variables (i.e., no array variables are used). Some sample simple expression are $-B, B + C, D$ or E , and F . We assume that all assignments in A are in one of the following forms:

- (a) $\langle \text{simple variable} \rangle := \langle \text{simple expression} \rangle$
- (b) $\langle \text{array variable} \rangle := \langle \text{simple variable} \rangle$
- (c) $\langle \text{simple variable} \rangle := \langle \text{array variable} \rangle$
- (d) $\langle \text{simple variable} \rangle := \text{Choice}(S)$, where S is a finite set $\{S_1, S_2, \dots, S_k\}$ or l, u . In the latter case the function chooses an integer in the range $[l : u]$.

Indexing within an array is done using a simple integer variable and all index values are positive. Only one-dimensional arrays are allowed. Clearly, all assignment statements not falling into one of the above categories can be replaced by a set of statements of these types. Hence, this restriction does not alter the class $\mathcal{NP}$.

3. All variables in A are of type integer or boolean.
4. Algorithm A contains no **read** or **write** statements. The only input to A is via its parameters. At the time A is invoked, all variables (other than the parameters) have value zero (or **false** if boolean).
5. Algorithm A contains no constants. Clearly, all constants in any algorithm can be replaced by new variables. These new variables can be added to the parameter list of A and the constants associated with them can be part of the input.
6. In addition to simple assignment statements, A is allowed to contain only the following types of statements:
 - (a) The statement **goto** k , where k is an instruction number.
 - (b) The statement **if** c **then goto** a ; Variable c is a simple boolean variable (i.e., not an array) and a is an instruction number.
 - (c) **Success()**, **Failure()**.
 - (d) Algorithm A may contain type declaration and dimension statements. These are not used during execution of A and so need not be translated into Q . The dimension information is used to allocate array space. It is assumed that successive elements in an array are assigned to consecutive words in memory.

It is assumed that the instructions in A are numbered sequentially from 1 to ℓ (if A has ℓ instructions). Every statement in A has a number. The **goto** instructions in (a) and (b) use this numbering scheme to effect a branch. It should be easy to see how to rewrite **repeat-until**, **for**, and so on, statements in terms of **goto** and **if c then goto a** statements. Also, note that the **goto k** statement can be replaced by the statement **if true then goto k** . So, this may also be eliminated.

7. Let $p(n)$ be a polynomial such that A takes no more than $p(n)$ time units on any input of length n . Because of the complexity assumption of 1), A cannot change or use more than $p(n)$ words of memory. We assume that A uses some subset of the words indexed $1, 2, 3, \dots, p(n)$. This assumption does not restrict the class of decision problems in $\mathcal{NP}$. To see this, let $f(1), f(2), \dots, f(k)$, $1 \leq k \leq p(n)$, be the distinct words used by A while working on input I . We can construct another polynomial time nondeterministic algorithm A' that uses $2p(n)$ words indexed $1, 2, \dots, 2p(n)$ and solves the same decision problem as A does. A' simulates the behavior of A . However, A' maps the addresses $f(1), f(2), \dots, f(k)$ onto the set $\{1, 2, \dots, k\}$. The mapping function used is determined dynamically and is stored as a table in words $p(n) + 1$ through $2p(n)$. If the entry at word $p(n) + i$ is j , then A' uses word i to hold the same value that A stored in word j . The simulation of A proceeds as follows: Let k be the number of distinct words referenced by A up to this time. Let j be a word referenced by A in the current step. A' searches its table to find word $p(n) + i$, $1 \leq i \leq k$, such that the contents of this word is j . If no such i exists, then A' sets $k := k + 1$; $i := k$; and word $p(n) + k$ is given the value j . A' makes use of the word i to do whatever A would have done with word j . Clearly, A' and A solve the same decision problem. The complexity of A' is $O(p^2(n))$ as it takes A' $p(n)$ time to search its table and simulate a step of A . Since $p^2(n)$ is also a polynomial in n , restricting our algorithms to use only consecutive words does not alter the classes $\mathcal{P}$ and $\mathcal{NP}$.

Formula Q makes use of several boolean variables. We state the semantics of two sets of variables used in Q :

1. $B(i, j, t)$, $1 \leq i \leq p(n)$, $1 \leq j \leq w$, $0 \leq t < p(n)$

$B(i, j, t)$ represents the status of bit j of word i following t steps (or time units) of computation. The bits in a word are numbered from right to left. The rightmost bit is numbered 1. Q is constructed so that in any truth assignment for which Q is true, $B(i, j, t)$ is true if and only if the corresponding bit has value 1 following t steps of some successful computation of A on input I .

2. $S(j, t)$, $1 \leq j \leq \ell$, $1 \leq t \leq p(n)$

Recall that ℓ is the number of instructions in A . $S(j, t)$ represents the instruction to be executed at time t . Q is constructed so that in any truth assignment for which Q is true, $S(j, t)$ is true if and only if the instruction executed by A at time t is instruction j .

Q is made up of six subformulas, C, D, E, F, G , and H . $Q = C \wedge D \wedge E \wedge F \wedge G \wedge H$. These subformulas make the following assertions:

C : The initial status of the $p(n)$ words represents the input I . All non-input variables are zero.

D : Instruction 1 is the first instruction to execute.

E : At the end of the i th step, there can be only one next instruction to execute. Hence, for any fixed i , exactly one of the $S(j, i)$, $1 \leq j \leq \ell$, can be true.

F : If $S(j, i)$ is true, then $S(j, i+1)$ is also true if instruction j is a Success or Failure statement. $S(j+1, i+1)$ is true if j is an assignment statement. If j is a **goto** k statement, then $S(k, i+1)$ is true. The last possibility for j is the **if** c **then** a ; statement. In this case $S(a, i+1)$ is true if c is true and $S(j+1, i+1)$ is true if c is false.

G : If the instruction executed at step t is not an assignment statement, then the $B(i, j, t)$'s are unchanged. If this instruction is an assignment and the variable on the left-hand side is X , then only X may change. This change is determined by the right-hand side of the instruction.

H : The instruction to be executed at time $p(n)$ is a Success instruction. Hence the computation terminates successfully.

Clearly, if C through H make the above assertions, then $Q = C \wedge D \wedge E \wedge F \wedge G \wedge H$ is satisfiable if and only if there is a successful computation of A on input I . We now give the formulas C through H . While presenting these formulas, we also indicate how each may be transformed into CNF. This transformation increases the length of Q by an amount independent of n (but dependent on w and ℓ). This enables us to show that CNF-satisfiability is $\mathcal{NP}$ -complete.

1. Formula C describes the input I . We have

$$C = \bigwedge_{\substack{1 \leq i \leq p(n) \\ 1 \leq j \leq w}} T(i, j, 0)$$

$T(i, j, 0)$ is $B(i, j, 0)$ if the input calls for bit $B(i, j, 0)$ (i.e., bit j of word i) to be 1. $T(i, j, 0)$ is $\bar{B}(i, j, 0)$ otherwise. Thus, if there is no input, then

$$C = \bigwedge_{\substack{1 \leq i \leq p(n) \\ 1 \leq j \leq w}} \bar{B}(i, j, 0)$$

Clearly, C is uniquely determined by I and is in CNF. Also, C is satisfiable only by a truth assignment representing the initial values of all variables in A .

$$2. D = S(1, 1) \wedge \bar{S}(2, 1) \wedge \bar{S}(3, 1) \wedge \cdots \wedge \bar{S}(\ell, 1)$$

Clearly, D is satisfiable only by the assignment $S(1, 1) = \text{true}$ and $S(i, 1) = \text{false}$, $2 \leq i \leq \ell$. Using our interpretation of $S(i, 1)$, this means that D is true if and only if instruction 1 is the first to be executed. Note that D is in CNF.

$$3. E = \bigwedge_{1 < t \leq p(n)} E_t$$

Each E_t will assert that there is a unique instruction for step t . We can define E_t to be

$$E_t = (S(1, t) \vee (S(2, t) \vee \cdots \vee S(\ell, t))) \wedge \bigwedge_{\substack{1 \leq j \leq \ell \\ 1 \leq k \leq \ell \\ j \neq k}} (\bar{S}(j, t) \vee \bar{S}(k, t))$$

One can verify that E_t is true iff exactly one of the $S(j, t)$'s, $1 \leq j \leq \ell$, is true. Also, note that E is in CNF.

$$4. F = \bigwedge_{\substack{1 \leq i \leq \ell \\ 1 \leq t < p(n)}} F_{i,t}$$

Each $F_{i,t}$ asserts that either instruction i is not the one to be executed at time t or, if it is, then the instruction to be executed at time $t + 1$ is correctly determined by instruction i . Formally, we have

$$F_{i,t} = \bar{S}(i, t) \vee L$$

where L is defined as follows:

- (a) If instruction i is **Success** or **Failure**, then L is $S(i, t + 1)$. Hence the program cannot leave such an instruction.
- (b) If instruction i is **goto** k , then L is $S(k, t + 1)$.
- (c) If instruction i is **if** X **then goto** k and variable X is represented by word j , then L is $((B(j, 1, t - 1) \wedge S(k, t + 1)) \vee (\bar{B}(j, 1, t - 1) \wedge S(i + 1, t + 1)))$. This assumes that bit 1 of X is 1 if and only if X is true.

(d) If instruction i is not any of the above, then L is $S(i + 1, t + 1)$.

The $F_{i,t}$'s defined in cases (a), (b), and (d) are in CNF. The $F_{i,t}$ in case (c) can be transformed into CNF using the boolean identity $a \vee (b \wedge c) \vee (d \wedge e) \equiv (a \vee b \vee d) \wedge (a \vee c \vee d) \wedge (a \vee b \vee e) \wedge (a \vee c \vee e)$.

$$5. G = \bigwedge_{\substack{1 \leq i \leq \ell \\ 1 \leq t \leq p(n)}} G_{i,t}$$

Each $G_{i,t}$ asserts that at time t either instruction i is not executed or it is and the status of the $p(n)$ words after step t is correct with respect to the status before step t and the changes resulting from instruction i . Formally, we have

$$G_{i,t} = \bar{S}(i, t) \vee M$$

where M is defined as follows:

- (a) If instruction i is a **goto**, **if-then-goto**, **Success**, or **Failure** statement, then M asserts that the status of the $p(n)$ words is unchanged; that is, $B(k, j, t - 1) = B(k, j, t)$, $1 \leq k \leq p(n)$, $1 \leq j \leq w$.

$$\begin{aligned} M = & \bigwedge_{\substack{1 \leq k \leq p(n) \\ 1 \leq j \leq w}} ((B(k, j, t - 1) \\ & \wedge B(k, j, t)) \vee (\bar{B}(k, j, t - 1) \wedge \bar{B}(k, j, t))) \end{aligned}$$

In this case, $G_{i,t}$ can be written as

$$\begin{aligned} G_{i,t} = & \bigwedge_{\substack{1 \leq k \leq p(n) \\ 1 \leq j \leq w}} (\bar{S}(i, t) \\ & \vee (B(k, j, t - 1) \wedge B(k, j, t)) \\ & \vee (\bar{B}(k, j, t - 1) \wedge \bar{B}(k, j, t))) \end{aligned}$$

Each clause in $G_{i,t}$ is of the form $z \vee (x \wedge s) \vee (\bar{x} \wedge \bar{s})$, where z is $\bar{S}(i, t)$, x represents a $B(., t - 1)$, and s represents a $B(., t)$. Note that $z \vee (x \wedge s) \vee (\bar{x} \wedge \bar{s})$ is equivalent to $(x \vee \bar{s} \vee z) \wedge (\bar{x} \vee s \vee z)$. Hence, $G_{i,t}$ can be transformed into CNF easily.

- (b) If i is an assignment statement of type 2(a), then M depends on the operator (if any) on the right-hand side. We first describe the form of M for the case in which instruction i is of type $Y := V + Z$. Let Y, V , and Z be respectively represented in words y, v , and z . We make the simplifying assumption that all numbers are nonnegative. The exercises examine the case in which negative numbers

are allowed and 1's complement arithmetic is used. To get a formula asserting that the bits $B(y, j, t)$, $1 \leq j \leq w$, represent the sum of $B(v, j, t-1)$ and $B(z, j, t-1)$, $1 \leq j \leq w$, we have to make use of w additional bits $C(j, t)$, $1 \leq j \leq w$. $C(j, t)$ represents the carry from the addition of the bits $B(v, j, t-1)$, $B(z, j, t-1)$, and $C(j-1, t)$, $1 < j \leq w$. $C(1, t)$ is the carry from the addition of $B(v, 1, t-1)$ and $B(z, 1, t-1)$. Recall that a bit is 1 iff the corresponding variable is true. Performing a bitwise addition of V and Z , we obtain $C(1, t) = B(v, 1, t-1) \wedge B(z, 1, t-1)$ and $B(y, 1, t) = B(v, 1, t-1) \oplus B(z, 1, t-1)$, where $\oplus$ is the **exclusive or** operation ($a \oplus b$ is true iff exactly one of a and b is true). Note that $a \oplus b \equiv (a \vee b) \wedge (\overline{a \wedge b}) \equiv (a \vee b) \wedge (\bar{a} \vee \bar{b})$. Hence, the right-hand side of the expression for $B(y, 1, t)$ can be transformed into CNF using this identity. For the other bits of Y , one can verify that

$$B(y, j, t) = B(v, j, t-1) \oplus (B(z, j, t-1) \oplus C(j-1, t)) \quad \text{and}$$

$$\begin{aligned} C(j, t) &= (B(v, j, t-1) \wedge B(z, j, t-1)) \vee (B(v, j, t-1) \\ &\quad \wedge C(j-1, t)) \\ &\quad \vee (B(z, j, t-1) \wedge C(j-1, t)) \end{aligned}$$

Finally, we require that $C(w, t) = \text{false}$ (i.e., there is no overflow). Let M' be the **and** of all the equations for $B(y, j, t)$ and $C(j, t)$, $1 \leq j \leq w$. M is given by

$$\begin{aligned} M &= \left(\bigwedge_{\substack{1 \leq k \leq p(n) \\ k \neq y \\ 1 \leq j \leq w}} ((B(k, j, t-1) \wedge B(k, j, t)) \right. \\ &\quad \left. \wedge (\bar{B}(k, j, t-1) \wedge \bar{B}(k, j, t))) \right) \wedge M' \end{aligned}$$

$G_{i,t}$ can be converted into CNF using the idea of 5(a). This transformation increases the length of $G_{i,t}$ by a constant factor independent of n . We leave it to the reader to figure out what M is when instruction i is either of the forms $Y := V$; and $Y := V \odot Z$; for $\odot$ one of $-, /, *, <, >, \leq, =$, and so on.

When i is an assignment statement of type 2(b) or 2(c), then it is necessary to select the correct array element. Consider an instruction of type 2(b): $R[m] := X$;. In this case formula M can be written as

$$M = W \wedge \left(\bigwedge_{1 \leq j \leq u} M_j \right)$$

where u is the dimension of R . Note that because of restriction (7) on algorithm A , $u \leq p(n)$. W asserts that $1 \leq m \leq u$. The specification of W is left as an exercise. Each M_j asserts that either $m \neq j$ or $m = j$ and only the j th element of R changes. Let us assume that the values of X and m are respectively stored in words x and m and that $R(1 : u)$ is stored in words $\alpha, \alpha + 1, \dots, \alpha + u - 1$. M_j is given by

$$M_j = \bigvee_{1 \leq k \leq w} T(m, k, t - 1) \vee Z$$

where T is B if the k th bit in the binary representation of j is 0 and T is $\bar{B}$ otherwise. Z is defined as

$$\begin{aligned} Z = \bigwedge_{\substack{1 \leq k \leq w \\ 1 \leq r \leq p(n) \\ r \neq \alpha + j - 1}} ((B(r, k, t - 1) \wedge B(r, k, t)) \vee (\bar{B}(r, k, t - 1) \\ \wedge \bar{B}(r, k, t - 1))) \\ \bigwedge_{1 \leq k \leq w} ((B(\alpha + j - 1, k, t) \wedge B(x, k, t - 1)) \\ \vee (\bar{B}(\alpha + j - 1, k, t) \wedge \bar{B}(x, k, t - 1))) \end{aligned}$$

Note that the number of literals in M is $O(p^2(n))$. Since j is w bits long, it can represent only numbers smaller than 2^w . Hence, for $u \geq 2^w$, we need a different indexing scheme. A simple generalization is to allow multiprecision arithmetic. The index variable j can then use as many words as needed. The number of words used depends on u . At most $\log(p(n))$ words are needed. This calls for a slight change in M_j , but the number of literals in M remains $O(p^2(n))$. There is no need to explicitly incorporate multiprecision arithmetic as by giving the program access to individual words in a multiprecision index j , we can require the program to simulate multiprecision arithmetic.

When i is an instruction of type 2(c), the form of M is similar to that obtained for instructions of type 2(b). Next, we describe how to construct M for the case in which i is of the form $Y := \text{Choice}(S);$, where S is either a set of the form $S = \{S_1, S_2, \dots, S_k\}$ or S is of the form r, u . Assume Y is represented by word y . If S is a set, then we define

$$M = \bigvee_{1 \leq j \leq k} M_j$$

M_j asserts that Y is S_j . This is easily done by choosing $M_j = a_1 \wedge a_2 \wedge \dots \wedge a_w$, where $a_\ell = B(y, \ell, t)$ if bit ℓ is 1 in S_ℓ and $a_\ell = \bar{B}(y, \ell, t)$ if bit ℓ is zero in S_ℓ . If S is of the form r, u , then M is just the formula

that asserts $r \leq Y \leq u$. This is left as an exercise. In both cases, $G_{i,t}$ can be transformed into CNF and the length of $G_{i,t}$ increased by at most a constant amount.

6. Let $i_1, i_2, \dots, i_k$ be the statement numbers corresponding to success statements in A . H is given by

$$H = S(i_1, p(n)) \vee S(i_2, p(n)) \vee \dots \vee S(i_k, p(n))$$

One can readily verify that $Q = C \wedge D \wedge E \wedge F \wedge G \wedge H$ is satisfiable if and only if the computation of algorithm A with input I terminates successfully. Further, Q can be transformed into CNF as described above. Formula C contains $wp(n)$ literals, D contains ℓ literals, E contains $O(\ell^2 p(n))$ literals, F contains $O(\ell p(n))$ literals, G contains $O(\ell w p^3(n))$ literals, and H contains at most ℓ literals. The total number of literals appearing in Q is $O(\ell w p^3(n)) = O(p^3(n))$ as ℓw is constant. Since there are $O(wp^2(n) + \ell p(n))$ distinct literals in Q , each literal can be written using $O(\log(wp^2(n) + \ell p(n))) = O(\log n)$ bits. The length of Q is therefore $O(p^3(n) \log n) = O(p^4(n))$ as $p(n)$ is at least n . The time to construct Q from A and I is also $O(p^3(n) \log n)$.

The preceding construction shows that every problem in $\mathcal{NP}$ reduces to satisfiability and also to CNF-satisfiability. Hence, if either of these two problems is in $\mathcal{P}$, then $\mathcal{NP} \subseteq \mathcal{P}$ and so $\mathcal{P} = \mathcal{NP}$. Also, since satisfiability is in $\mathcal{NP}$, the construction of a CNF formula Q shows that satisfiability $\propto$ CNF-satisfiability. This together with the knowledge that CNF-satisfiability is in $\mathcal{NP}$ implies that CNF-satisfiability is $\mathcal{NP}$ -complete. Note that satisfiability is also $\mathcal{NP}$ -complete as satisfiability $\propto$ satisfiability and satisfiability is in $\mathcal{NP}$.

EXERCISES

1. In conjunction with formula G in the proof of Cook's theorem (Section 11.2), obtain M for the following cases for instruction i . Note that M can contain at most $O(p(n))$ literals (as a function of n). Obtain M under the assumption that negative numbers are represented in ones complement. Show how the corresponding $G_{i,t}$'s can be transformed into CNF. The length of $G_{i,t}$ must increase by no more than a constant factor (say w^2) during this transformation.
 - (a) $Y := Z$;
 - (b) $Y := V - Z$;
 - (c) $Y := V + Z$;
 - (d) $Y := V * Z$;

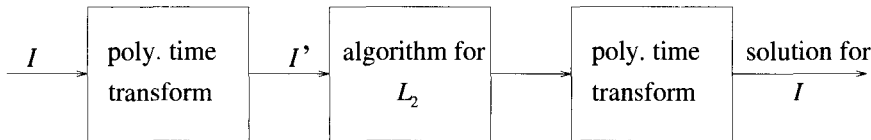


Figure 11.3 Reduction of L_1 to L_2

- (e) $Y := \text{Choice}(0, 1);$
 - (f) $Y := \text{Choice}(r, u);$, where r and u are variables
2. Show how to encode the following instructions as CNF formulas: (a) **for** and (b) **while**.
 3. Prove or disprove: If there exists a polynomial time algorithm to convert a boolean formula in CNF into an equivalent formula in DNF, then $\mathcal{P} = \mathcal{NP}$.

11.3 $\mathcal{NP}$ -HARD GRAPH PROBLEMS

The strategy we adopt to show that a problem L_2 is $\mathcal{NP}$ -hard is:

1. Pick a problem L_1 already known to be $\mathcal{NP}$ -hard.
2. Show how to obtain (in polynomial deterministic time) an instance I' of L_2 from any instance I of L_1 such that from the solution of I' we can determine (in polynomial deterministic time) the solution to instance I of L_1 (see Figure 11.3).
3. Conclude from step (2) that $L_1 \propto L_2$.
4. Conclude from steps (1) and (3) and the transitivity of $\propto$ that L_2 is $\mathcal{NP}$ -hard.

For the first few proofs we go through all the above steps. Later proofs explicitly deal only with steps (1) and (2). An $\mathcal{NP}$ -hard decision problem L_2 can be shown to be $\mathcal{NP}$ -complete by exhibiting a polynomial time non-deterministic algorithm for L_2 . All the $\mathcal{NP}$ -hard decision problems we deal with here are $\mathcal{NP}$ -complete. The construction of polynomial time nondeterministic algorithms for these problems is left as an exercise.

11.3.1 Clique Decision Problem (CDP)

The clique decision problem was introduced in Section 11.1. We show in Theorem 11.2 that $\text{CNF-satisfiability} \propto \text{CDP}$. Using this result, the transitivity of $\propto$, and the knowledge that $\text{satisfiability} \propto \text{CNF-satisfiability}$ (Section 11.2), we can readily establish that $\text{satisfiability} \propto \text{CDP}$. Hence, CDP is $\mathcal{NP}$ -hard. Since, $\text{CDP} \in \mathcal{NP}$, CDP is also $\mathcal{NP}$ -complete.

Theorem 11.2 $\text{CNF-satisfiability} \propto \text{clique decision problem}$.

Proof: Let $F = \bigwedge_{1 \leq i \leq k} C_i$ be a propositional formula in CNF. Let x_i , $1 \leq i \leq n$, be the variables in F . We show how to construct from F a graph $G = (V, E)$ such that G has a clique of size at least k if and only if F is satisfiable. If the length of F is m , then G is obtainable from F in $O(m)$ time. Hence, if we have a polynomial time algorithm for CDP, then we can obtain a polynomial time algorithm for CNF-satisfiability using this construction.

For any F , $G = (V, E)$ is defined as follows: $V = \{\langle \sigma, i \rangle \mid \sigma \text{ is a literal in clause } C_i\}$ and $E = \{(\langle \sigma, i \rangle, \langle \delta, j \rangle) \mid i \neq j \text{ and } \sigma \neq \bar{\delta}\}$. A sample construction is given in Example 11.11.

Claim: F is satisfiable if and only if G has a clique of size $\geq k$.

Proof of Claim: If F is satisfiable, then there is a set of truth values for x_i , $1 \leq i \leq n$, such that each clause is true with this assignment. Thus, with this assignment there is at least one literal σ in each C_i such that σ is true. Let $S = \{\langle \sigma, i \rangle \mid \sigma \text{ is true in } C_i\}$ be a set containing exactly one $\langle \sigma, i \rangle$ for each i . Between any two nodes $\langle \sigma, i \rangle$ and $\langle \delta, j \rangle$ in S there is an edge in G , since $i \neq j$ and both σ and δ have the value true. Thus, S forms a clique in G of size k .

Similarly, if G has a clique $K = (V', E')$ of size at least k , then let $S = \{\langle \sigma, i \rangle \mid \langle \sigma, i \rangle \in V'\}$. Clearly, $|S| = k$ as G has no clique of size more than k . Furthermore, if $S' = \{\sigma \mid \langle \sigma, i \rangle \in S \text{ for some } i\}$, then S' cannot contain both a literal δ and its complement $\bar{\delta}$ as there is no edge connecting $\langle \delta, i \rangle$ and $\langle \bar{\delta}, j \rangle$ in G . Hence by setting $x_i = \text{true}$ if $x_i \in S'$ and $x_i = \text{false}$ if $\bar{x}_i \in S'$ and choosing arbitrary truth values for variables not in S' , we can satisfy all clauses in F . Hence, F is satisfiable iff G has a clique of size at least k . $\square$

Example 11.11 Consider $F = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$. The construction of Theorem 11.2 yields the graph of Figure 11.4. This graph contains six cliques of size two. Consider the clique with vertices $\{\langle x_1, 1 \rangle, \langle \bar{x}_2, 2 \rangle\}$. By setting $x_1 = \text{true}$ and $\bar{x}_2 = \text{true}$ (that is, $x_2 = \text{false}$), F is satisfied. The x_3 may be set either to true or false. $\square$

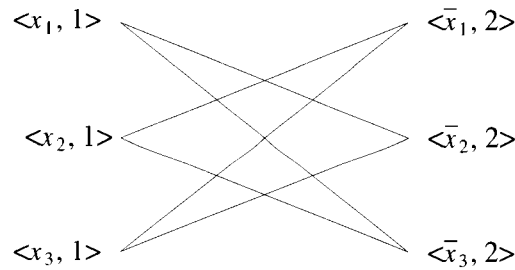


Figure 11.4 A sample graph and satisfiability

11.3.2 Node Cover Decision Problem (NCDP)

A set $S \subseteq V$ is a *node cover* for a graph $G = (V, E)$ if and only if all edges in E are incident to at least one vertex in S . The size $|S|$ of the cover is the number of vertices in S .

Example 11.12 Consider the graph of Figure 11.5. $S = \{2, 4\}$ is a node cover of size 2. $S = \{1, 3, 5\}$ is a node cover of size 3. $\square$

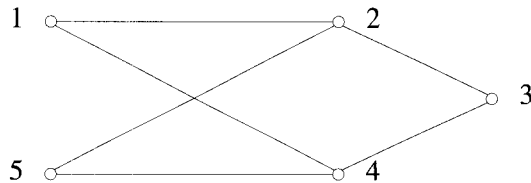


Figure 11.5 A sample graph and node cover

In the node cover decision problem we are given a graph G and an integer k . We are required to determine whether G has a node cover of size at most k .

Theorem 11.3 The clique decision problem $\propto$ the node cover decision problem.

Proof: Let $G = (V, E)$ and k define an instance of CDP. Assume that $|V| = n$. We construct a graph G' such that G' has a node cover of size at

most $n - k$ if and only if G has a clique of size at least k . Graph G' is given by $G' = (V, \bar{E})$, where $\bar{E} = \{(u, v) \mid u \in V, v \in V \text{ and } (u, v) \notin E\}$. The set G' is known as the *complement* of G .

Now, we show that G has a clique of size at least k if and only if G' has a node cover of size at most $n - k$. Let K be any clique in G . Since there are no edges in $\bar{E}$ connecting vertices in K , the remaining $n - |K|$ vertices in G' must cover all edges in $\bar{E}$. Similarly, if S is a node cover of G' , then $V - S$ must form a complete subgraph in G .

Since G' can be obtained from G in polynomial time, CDP can be solved in polynomial deterministic time if we have a polynomial time deterministic algorithm for NCDP. $\square$

Example 11.13 Figure 11.6 shows a graph G and its complement G' . In this figure, G' has a node cover of $\{4, 5\}$, since every edge of G' is incident either on the node 4 or on the node 5. Thus, G has a clique of size $5 - 2 = 3$ consisting of the nodes 1, 2, and 3. $\square$

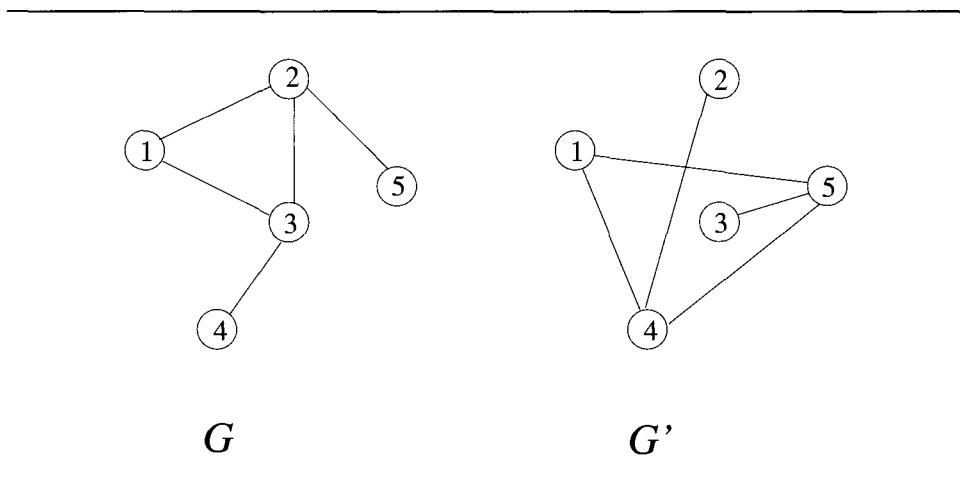


Figure 11.6 A graph and its complement

Note that since $\text{CNF-satisfiability} \propto \text{CDP}$, $\text{CDP} \propto \text{NCDP}$ and $\propto$ is transitive, it follows that NCDP is $\mathcal{NP}$ -hard. NCDP is also in $\mathcal{NP}$ because we can nondeterministically choose a subset $C \subseteq V$ of size k and verify in polynomial time that C is a cover of G . So NCDP is $\mathcal{NP}$ -complete.

11.3.3 Chromatic Number Decision Problem (CNDP)

A coloring of a graph $G = (V, E)$ is a function $f : V \rightarrow \{1, 2, \dots, k\}$ defined for all $i \in V$. If $(u, v) \in E$, then $f(u) \neq f(v)$. The *chromatic number decision problem* is to determine whether G has a coloring for a given k .

Example 11.14 A possible 2-coloring of the graph of Figure 11.5 is $f(1) = f(3) = f(5) = 1$ and $f(2) = f(4) = 2$. Clearly, this graph has no 1-coloring. $\square$

In proving CNDP to be $\mathcal{NP}$ -hard, we shall make use of the $\mathcal{NP}$ -hard problem SATY. This is the CNF-satisfiability problem with the restriction that each clause has at most three literals. The reduction CNF-satisfiability $\propto$ SATY is left as an exercise.

Theorem 11.4 Satisfiability with at most three literals per clause $\propto$ chromatic number decision problem.

Proof: Let F be a CNF formula having at most three literals per clause and having r clauses $C_1, C_2, \dots, C_r$. Let x_i , $1 \leq i \leq n$, be the n variables in F . We can assume $n \geq 4$. If $n < 4$, then we can determine whether F is satisfiable by trying out all eight possible truth value assignments to x_1, x_2 , and x_3 . We construct, in polynomial time, a graph G that is $n + 1$ colorable if and only if F is satisfiable. The graph $G = (V, E)$ is defined by

$$V = \{x_1, x_2, \dots, x_n\} \cup \{\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n\} \cup \{y_1, y_2, \dots, y_n\} \cup \{C_1, C_2, \dots, C_r\}$$

where $y_1, y_2, \dots, y_n$ are new variables, and

$$E = \{(x_i, \bar{x}_i), 1 \leq i \leq n\} \cup \{(y_i, y_j) | i \neq j\} \cup \{(y_i, x_j) | i \neq j\} \\ \cup \{(y_i, \bar{x}_j) | i \neq j\} \cup \{(x_i, C_j) | x_i \notin C_j\} \cup \{\bar{x}_i, C_j | \bar{x}_i \notin C_j\}$$

To see that G is $n + 1$ colorable if and only if F is satisfiable, we first observe that the y_i 's form a complete subgraph on n vertices. Hence, each y_i must be assigned a distinct color. Without loss of generality we can assume that in any coloring of G , y_i is given the color i . Since y_i is also connected to all the x_j 's and $\bar{x}_j$'s except x_i and $\bar{x}_i$, the color i can be assigned to only x_i and $\bar{x}_i$. However, $(x_i, \bar{x}_i) \in E$ and so a new color, $n + 1$, is needed for one of these vertices. The vertex that is assigned the new color $n + 1$ is called a *false vertex*. The other vertex is a *true vertex*. The only way to color G using $n + 1$ colors is to assign color $n + 1$ to one of $\{x_i, \bar{x}_i\}$ for each i , $1 \leq i \leq n$.

Under what conditions can the remaining vertices be colored using no new colors? Since $n \geq 4$ and each clause has at most three literals, each C_i is adjacent to a pair of vertices $x_j, \bar{x}_j$ for at least one j . Consequently,

no C_i can be assigned the color $n + 1$. Also, no C_i can be assigned a color corresponding to an x_j or $\bar{x}_j$ not in clause C_i . The last two statements imply that the only colors that can be assigned to C_i correspond to vertices x_j or $\bar{x}_j$ that are in clause C_i and are true vertices. Hence, G is $n + 1$ colorable if and only if there is a true vertex corresponding to each C_i . So, G is $n + 1$ colorable iff F is satisfiable. $\square$

11.3.4 Directed Hamiltonian Cycle (DHC) (*)

A directed Hamiltonian cycle in a directed graph $G = (V, E)$ is a directed cycle of length $n = |V|$. So, the cycle goes through every vertex exactly once and then returns to the starting vertex. The DHC problem is to determine whether G has a directed Hamiltonian cycle.

Example 11.15 1, 2, 3, 4, 5, 1 is a directed Hamiltonian cycle in the graph of Figure 11.7. If the edge $\langle 5, 1 \rangle$ is deleted from this graph, then it has no directed Hamiltonian cycle. $\square$

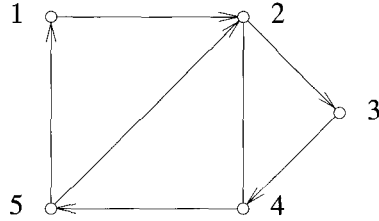


Figure 11.7 A sample graph and Hamiltonian cycle

Theorem 11.5 CNF-satisfiability $\propto$ directed Hamiltonian cycle.

Proof: Let F be a propositional formula in CNF. We show how to construct a directed graph G such that F is satisfiable if and only if G has a directed Hamiltonian cycle. Since this construction can be carried out in time polynomial in the size of F , it will follow that CNF-satisfiability $\propto$ DHC. Understanding the construction of G is greatly facilitated by the use of an example. The example we use is $F = C_1 \wedge C_2 \wedge C_3 \wedge C_4$, where

$$\begin{aligned} C_1 &= x_1 \vee \bar{x}_2 \vee x_4 \vee \bar{x}_5 \\ C_2 &= \bar{x}_1 \vee x_2 \vee x_3 \\ C_3 &= \bar{x}_1 \vee \bar{x}_3 \vee x_5 \\ C_4 &= \bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3 \vee x_4 \vee \bar{x}_5 \end{aligned}$$

Assume that F has r clauses $C_1, C_2, \dots, C_r$ and n variables $x_1, x_2, \dots, x_n$. Draw an array with r rows and $2n$ columns. Row i denotes clause C_i . Each variable x_i is represented by two adjacent columns, one for each of the literals x_i and $\bar{x}_i$. Figure 11.8 shows the array for the example formula. Insert a $\odot$ into column x_i and row C_j if and only if x_i is a literal in C_j . Insert a $\odot$ into column $\bar{x}_i$ and row C_j if and only if $\bar{x}_i$ is a literal in C_j . Between each pair of columns x_i and $\bar{x}_i$ introduce two vertices u_i and v_i at the top and v_i at the bottom of the column. For each i , draw two chains of edges upward from v_i to u_i , one connecting together all $\odot$ s in column x_i and the other connecting all $\odot$ s in column $\bar{x}_i$ (see Figure 11.8). Now, draw edges $\langle u_i, v_{i+1} \rangle$, $1 \leq i < n$. Introduce a box $\boxed{i}$ at the right end of each row C_i , $1 \leq i < r$. Draw the edges $\langle u_n, \boxed{1} \rangle$ and $\langle \boxed{r}, v_1 \rangle$. Draw edges $\langle \boxed{i}, \boxed{i+1} \rangle$, $1 \leq i < r$ (see Figure 11.8).

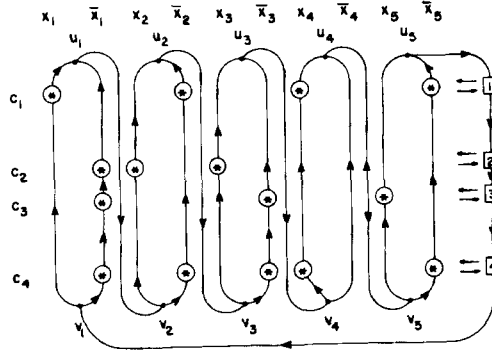


Figure 11.8 Array structure for the formula in Theorem 11.5

To complete the graph, we replace each $\odot$ and $\boxed{i}$ by a subgraph. Each $\odot$ is replaced by the subgraph of Figure 11.9(a) (of course, unique vertex labelings are needed for each copy of the subgraph). Each box $\boxed{i}$ is replaced by the subgraph of Figure 11.10. In this subgraph A_i is an entrance node and B_i an exit node. The edges $\langle \boxed{i}, \boxed{i+1} \rangle$ referred to earlier are really $\langle B_i, A_{i+1} \rangle$. Edge $\langle u_n, \boxed{1} \rangle$ is $\langle u_n, A_1 \rangle$ and $\langle \boxed{r}, v_1 \rangle$ is $\langle B_r, v_1 \rangle$. The variable j_i is the number of literals in clause C_i . In the subgraph of Figure 11.10 an edge of the type shown in Figure 11.11 indicates a connection to a $\odot$ subgraph in row C_i . $R_{i,a}$ is connected to the 1 vertex of the $\odot$ and $R_{i,a+1}$ (or $R_{i,1}$ if $a = j_i$) is entered from the 3 vertex.

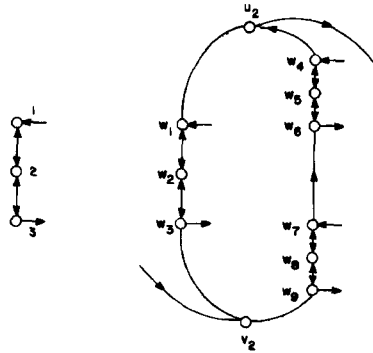


Figure 11.9 The $\odot$ subgraph and its insertion into column 2

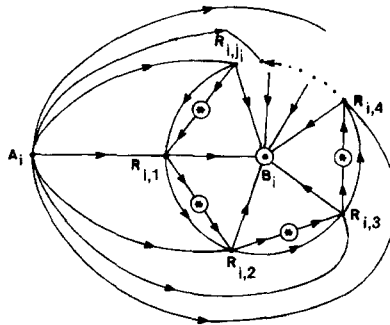


Figure 11.10 The H_i subgraph

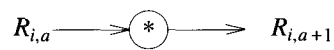


Figure 11.11 A construct in the proof of Theorem 11.5

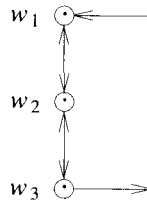


Figure 11.12 Another construct in the proof of Theorem 11.5

Thus in the $\odot$ subgraph (shown in Figure 11.12) of Figure 11.9(b) w_1 and w_3 are the 1 and 3 vertices respectively. The incoming edge is $\langle R_{i,1}, w_1 \rangle$ and the outgoing edge is $\langle w_3, R_{i,2} \rangle$. This completes the construction of G .

If F is satisfiable, then let S be an assignment of truth values for which F is true. A Hamiltonian cycle for G can start at v_1 and go to u_1 , then to v_2 , then to u_2 , then to v_3 , then to $u_3, \dots$, and then to u_n . In going from v_i to u_i , this cycle uses the column corresponding to x_i if x_i is true in S . Otherwise it goes up the column corresponding to $\bar{x}_i$. From u_n this cycle goes to A_1 and then through $R_{1,1}, R_{1,2}, R_{1,3}, \dots, R_{1,j_i}$, and B_1 to A_2 to $\dots$ to v_1 . In going from $R_{i,a}$ to $R_{i,a+1}$ in any subgraph $\boxed{i}$, a diversion is made to a $\odot$ subgraph in row i if and only if the vertices of that $\odot$ subgraph are not already on the path from v_i to $R_{i,a}$. Note that if C_i has i_j literals, then the construction of $\boxed{i}$ allows a diversion to at most $i_j - 1$ $\odot$ subgraphs. This is adequate as at least one $\odot$ subgraph must already have been traversed in row C_i (because at least one such subgraph must correspond to a true literal). So, if F is satisfiable, then G has a directed Hamiltonian cycle.

It remains to show that if G has a directed Hamiltonian cycle, then F is satisfiable. This can be seen by starting at vertex v_1 on any Hamiltonian cycle for G . Because of the construction of the $\odot$ and $\boxed{i}$ subgraphs, such a cycle must proceed by going up exactly one column of each pair $(x_i, \bar{x}_i)$. In addition, this part of the cycle must traverse at least one $\odot$ subgraph in each row. Hence the columns used in going from v_i to u_i , $1 \leq i \leq n$, define a truth assignment for which F is true.

We conclude that F is satisfiable if and only if G has a Hamiltonian cycle. The theorem now follows from the observation that G can be obtained from F in polynomial time. $\square$

11.3.5 Traveling Salesperson Decision Problem (TSP)

The traveling salesperson problem was introduced in Chapter 5. The corresponding decision problem is to determine whether a complete directed

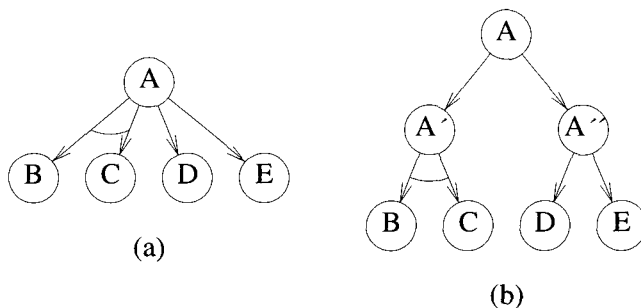


Figure 11.13 Graphs representing problems

graph $G = (V, E)$ with edge costs $c(u, v)$ has a tour of cost at most M .

Theorem 11.6 Directed Hamiltonian cycle (DHC) $\propto$ the traveling salesperson decision problem (TSP).

Proof: From the directed graph $G = (V, E)$ construct the complete directed graph $G' = (V, E')$, $E' = \{\langle i, j \rangle \mid i \neq j\}$ and $c(i, j) = 1$ if $\langle i, j \rangle \in E$; $c(i, j) = 2$ if $i \neq j$ and $\langle i, j \rangle \notin E$. Clearly, G' has a tour of cost at most n iff G has a directed Hamiltonian cycle. $\square$

11.3.6 AND/OR Graph Decision Problem (AOG)

Many complex problems can be broken down into a series of subproblems such that the solution of all or some of these results in the solution of the original problem. These subproblems can be broken down further into sub-subproblems, and so on, until the only problems remaining are sufficiently primitive as to be trivially solvable. This breaking down of a complex problem into several subproblems can be represented by a directed graphlike structure in which nodes represent problems and descendants of nodes represent the subproblems associated with them.

Example 11.16 The graph of Figure 11.13(a) represents a problem A that can be solved by solving either both the subproblems B and C or the single subproblem D or E . $\square$

Groups of subproblems that must be solved in order to imply a solution to the parent node are joined together by an arc going across the respective edges (as the arc across the edges $\langle A, B \rangle$ and $\langle A, C \rangle$). By introducing dummy

nodes in Figure 11.13(b), all nodes can be made to be such that their solution requires either all descendants to be solved or only one descendant to be solved. Nodes of the first type are called AND nodes and those of the latter type OR nodes. Nodes A and A'' of Figure 11.13(b) are OR nodes whereas node A' is an AND node. The AND nodes are drawn with an arc across all edges leaving the node. Nodes with no descendants are called *terminal*. Terminal nodes represent primitive problems and are marked either solvable or not solvable. Solvable terminal nodes are represented by rectangles. An AND/OR graph need not always be a tree.

Breaking down a problem into several subproblems is known as *problem reduction*. Problem reduction has been used on such problems as theorem proving, symbolic integration, and analysis of industrial schedules. When problem reduction is used, two different problems may generate a common subproblem. In this case it may be desirable to have only one node representing the subproblem (this would imply that the subproblem is to be solved only once). Figure 11.14 shows two AND/OR graphs for cases in which this is done.

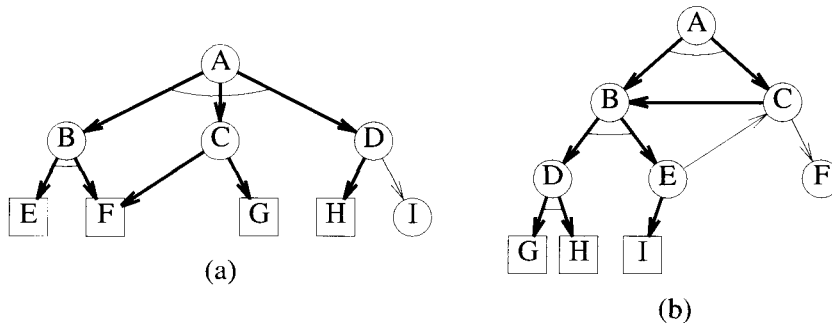


Figure 11.14 Two AND/OR graphs that are not trees

Note that the graph is no longer a tree. Furthermore, such graphs may have directed cycles as in Figure 11.14(b). The presence of a directed cycle does not in itself imply the unsolvability of the problem. In fact, problem A of Figure 11.14(b) can be solved by solving the primitive problems G, H, and I. This leads to the solution of D and E and hence of B and C. A *solution graph* is a subgraph of solvable nodes that shows that the problem is solved. Possible solution graphs for the graphs of Figure 11.14 are shown by heavy edges.

Let us assume that there is a cost associated with each edge in the AND/OR graph. The *cost* of a solution graph H of an AND/OR graph G is the sum of the costs of the edges in H . The *AND/OR graph decision*

problem (AOG) is to determine whether G has a solution graph of cost at most k , for k a given input.

Example 11.17 Consider the directed graph of Figure 11.15. The problem to be solved is P_1 . To do this, one can solve node P_2, P_3 , or P_7 , as P_1 is an OR node. The cost incurred is then either 2, 2, or 8 (i.e., cost in addition to that of solving one of P_2, P_3 , or P_7). To solve P_2 , both P_4 and P_5 have to be solved, as P_2 is an AND node. The total cost to do this is 2. To solve P_3 , we can solve either P_5 or P_6 . The minimum cost to do this is 1. Node P_7 is free. In this example, then, the optimal way to solve P_1 is to solve P_6 first, then P_3 , and finally P_1 . The total cost for this solution is 3. $\square$

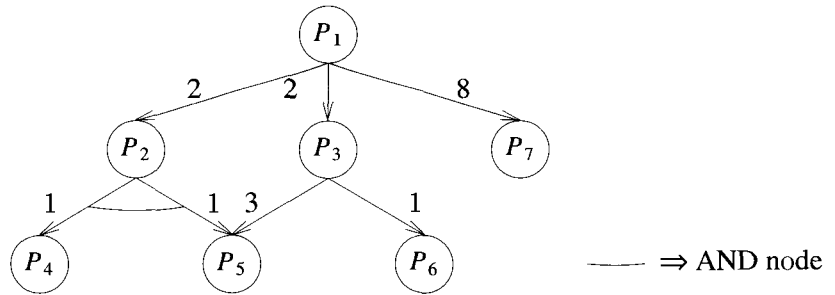


Figure 11.15 AND/OR graph

Theorem 11.7 CNF-satisfiability $\propto$ the AND/OR graph decision problem.

Proof: Let P be a propositional formula in CNF. We show how to transform a formula P in CNF into an AND/OR graph such that the AND/OR graph so obtained has a certain minimum cost solution if and only if P is satisfiable. Let

$$P = \bigwedge_{i=1}^k C_i, \quad C_i = \bigvee l_j$$

where the l_j 's are literals. The variables of P , $V(P)$ are $x_1, x_2, \dots, x_n$. The AND/OR graph will have nodes as follows:

1. There is a special node S with no incoming arcs. This node represents the problem to be solved.

2. The node S is an AND node with descendent nodes $P, x_1, x_2, \dots, x_n$.
3. Each node x_i represents the corresponding variable x_i in the formula P . Each x_i is an OR node with two descendents denoted Tx_i and Fx_i respectively. If Tx_i is solved, then this will correspond to assigning a truth value of true to the variable x_i . Solving node Fx_i will correspond to assigning a truth value of false to x_i .
4. The node P represents the formula P and is an AND node. It has k descendents $C_1, C_2, \dots, C_k$. Node C_i corresponds to the clause C_i in the formula P . The nodes C_i are OR nodes.
5. Each node of type Tx_i or Fx_i has exactly one descendent node that is terminal (i.e., has no edges leaving it). These terminal nodes are denoted $v_1, v_2, \dots, v_{2n}$.

To complete the construction of the AND/OR graph, the following edges and costs are added:

1. From each node C_i an edge $\langle C_i, Tx_j \rangle$ is added if x_j occurs in clause C_i . An edge $\langle C_i, Fx_j \rangle$ is added if $\bar{x}_j$ occurs in clause C_i . This is done for all variables x_j appearing in the clause C_i . Clause C_i is designated an OR node.
2. Edges from nodes of type Tx_i or Fx_i to their respective terminal nodes are assigned a weight, or cost of 1.
3. All other edges have a cost of 0.

In order to solve S , each of the nodes $P, x_1, x_2, \dots, x_n$ must be solved. Solving nodes $x_1, x_2, \dots, x_n$ costs n . To solve P , we must solve all the nodes $C_1, C_2, \dots, C_k$. The cost of a node C_i is at most 1. However, if one of its descendent nodes was solved while solving the nodes $x_1, x_2, \dots, x_n$, then the additional cost to solve C_i is 0, as the edges to its descendent nodes have cost 0 and one of its descendents has already been solved. That is, a node C_i can be solved at no cost if one of the literals occurring in the clause C_i has been assigned a value of true. From this it follows that the entire graph (that is, node S) can be solved at a cost n if there is some assignment of truth values to the x_i 's such that at least one literal in each clause is true under that assignment, i.e., if the formula P is satisfiable. If P is not satisfiable, then the cost is more than n .

We have now shown how to construct an AND/OR graph from a formula P such that the AND/OR graph so constructed has a solution of cost n if and only if P is satisfiable. Otherwise the cost is more than n . The construction clearly takes only polynomial time. This completes the proof. $\square$

Example 11.18 Consider the formula

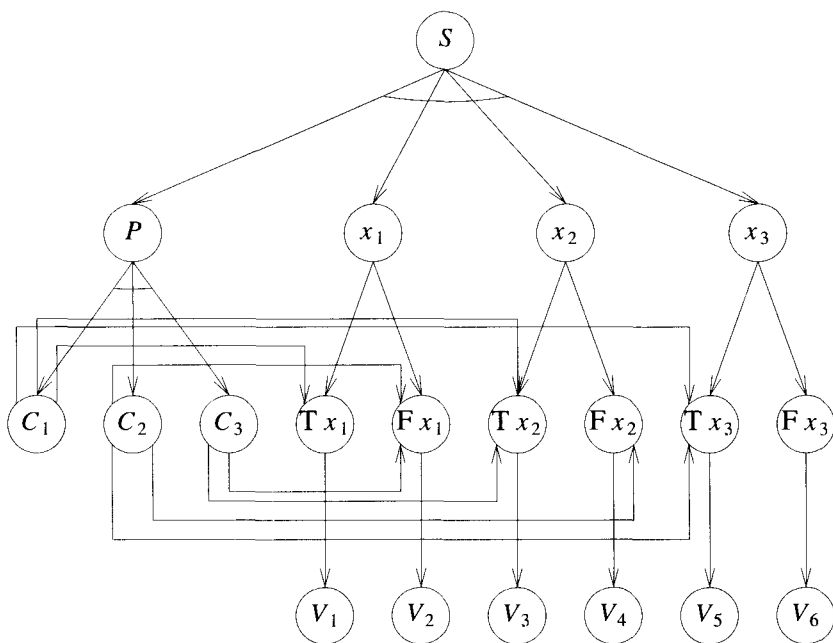
$$P = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2); \quad V(P) = x_1, x_2, x_3; \quad n = 3$$

Figure 11.16 shows the AND/OR graph obtained by applying the construction of Theorem 11.7.

The nodes Tx_1, Tx_2 , and Tx_3 can be solved at a total cost of 3. The node P costs nothing extra. The node S can then be solved by solving all its descendent nodes and the nodes Tx_1, Tx_2 , and Tx_3 . The total cost for this solution is 3 (which is n). Assigning the truth value of true to the variables of P results in P 's being true. $\square$

EXERCISES

1. Let SATY be the problem of determining whether a propositional formula in CNF having at most three literals per clause is satisfiable. Show that CNF-satisfiability $\propto$ SATY. *Hint:* Show how to write a clause with more than three literals as the **and** of several clauses each containing at most three literals. For this you have to introduce some new variables. Any assignment that satisfies the original clause must satisfy all the new clauses created.
2. Let SAT3 be similar to SATY (Exercise 1) except that each clause has exactly three literals. Show that SATY $\propto$ SAT3.
3. Let F be a propositional formula in CNF. Two literals x and y in F are *compatible* if and only if they are not in the same clause and $x \neq \bar{y}$. The literals x and y are *incompatible* if and only if x and y are not compatible. Let SATINC be the problem of determining whether a formula F in which each literal is incompatible with at most three other literals is satisfiable. Show that SAT3 $\propto$ SATINC.
4. Let 3-NODE COVER be the node cover decision problem of Section 11.3 restricted to graphs of degree 3. Show that SATINC $\propto$ 3-NODE COVER (see Exercise 3).
5. [Feedback node set]
 - (a) Let $G = (V, E)$ be a directed graph. Let $S \subseteq V$ be a subset of vertices such that the deletion of S and all edges incident to vertices in S results in a graph G' with no directed cycles. Such an S is a feedback node set. The size of S is the number of vertices in S . The feedback node set decision problem (FNS) is to determine for a given input k whether G has a feedback node set of size at most k . Show that the node cover decision problem $\propto$ FNS.



AND nodes joined by arc
All other nodes are OR

Figure 11.16 AND/OR graph for Example 11.18

- (b) Write a polynomial time nondeterministic algorithm for FNS.
- 6. [Feedback arc set] Let $G = (V, E)$ be a directed graph. $S \subseteq E$ is a feedback arc set of G if and only if every directed cycle in G contains an edge in S . The feedback arc set decision problem (FAS) is to determine whether G has a feedback arc set of size at most k .
 - (a) Show that the node cover decision problem $\propto$ FAS.
 - (b) Write a polynomial time nondeterministic algorithm for FAS.
- 7. The feedback node set optimization problem is to find a minimum feedback node set (see Exercise 5). Show that this problem reduces to FNS.
- 8. Show that the feedback arc set minimization problem reduces to FAS (Exercise 6).
- 9. [Hamiltonian cycle] Let UHC be the problem of determining whether in any given undirected graph G , there exists an undirected cycle going through each vertex exactly once and returning to the start vertex. Show that DHC $\propto$ UHC (DHC is defined in Section 11.3).
- 10. Show UHC $\propto$ CNF-satisfiability.
- 11. Show DHC $\propto$ CNF-satisfiability.
- 12. [Hamiltonian path] An i to j Hamiltonian path in graph G is a path from vertex i to vertex j that includes each vertex exactly once. Show that UHC is reducible to the problem of determining whether G has an i to j Hamiltonian path.
- 13. [Minimum equivalent graph] A directed graph $G = (V, E)$ is an equivalent graph of the directed graph $G' = (V, E')$ if and only if $E \subseteq E'$ and the transitive closures of G and G' are the same. G is a minimum equivalent graph if and only if $|E|$ is minimum among all equivalent graphs of G' . The minimum equivalent graph decision problem (MEG) is to determine whether G' has a minimum equivalent graph with $|E| \leq k$, where k is some given input.
 - (a) Show that DHC $\propto$ MEG.
 - (b) Write a nondeterministic polynomial time algorithm for MEG.
- 14. [Clique cover] The clique cover decision problem (CC) is to determine whether G is the union of l or fewer cliques. Show that the chromatic number decision problem $\propto$ CC.

15. [Set cover] Let $F = \{S_j\}$ be a finite family of sets. Let $T \subseteq F$ be a subset of F . T is a cover of F iff

$$\bigcup_{S_i \in T} S_i = \bigcup_{S_i \in F} S_i$$

The set cover decision problem is to determine whether F has a cover T containing no more than k sets. Show that the node cover decision problem is reducible to this problem.

16. [Exact cover] Let $F = \{S_j\}$ be as in Exercise 15. $T \subseteq F$ is an exact cover of F iff T is a cover of F and the sets in T are pairwise disjoint. Show that the chromatic number decision problem reduces to the problem of determining whether F has an exact cover.
17. Show that $\text{SAT3} \propto \text{EXACT COVER}$ (see Exercise 16).
18. [Hitting set] Let F be as in Exercise 16. The hitting set problem is to determine whether there exists a set H such that $|H \cap S_j| = 1$ for all $S_j \in F$. Show that exact cover $\propto$ hitting set.
19. [Tautology] A propositional formula is a tautology if and only if it is true for all possible truth assignments to its variables. The tautology problem is to determine whether a DNF formula is a tautology.
- Show that CNF-satisfiability $\propto$ DNF tautology.
 - Write a polynomial time nondeterministic algorithm $\text{TAUT}(F)$ that terminates successfully if and only if F is not a tautology.
20. [Minimum boolean form] Let the length of a propositional formula be equal to the sum of the number of literals in each clause. Two formulas F and G on variables $x_1, \dots, x_n$ are equivalent if for all assignments to $x_1, \dots, x_n$, F is true if and only if G is true. Show that deciding whether F has an equivalent formula of length no more than k is $\mathcal{NP}$ -hard. (*Hint:* Show DNF tautology reduces to this problem.)

11.4 $\mathcal{NP}$ -HARD SCHEDULING PROBLEMS

To prove the results of this section, we need to use the $\mathcal{NP}$ -hard problem called *partition*. This problem requires us to decide whether a given multi-set $A = \{a_1, a_2, \dots, a_n\}$ of n positive integers has a partition P such that $\sum_{i \in P} a_i = \sum_{i \notin P} a_i$. We can show this problem is $\mathcal{NP}$ -hard by first showing the sum of subsets problem (Chapter 7) to be $\mathcal{NP}$ -hard. Recall that in the sum of subsets problem we have to determine whether $A = \{a_1, a_2, \dots, a_n\}$ has a subset S that sums to a given integer M .

Theorem 11.8 Exact cover $\propto$ sum of subsets.

Proof: The exact cover problem is shown $\mathcal{NP}$ -hard in Section 11.3, Exercise 16. In this problem we are given a family of sets $F = \{S_1, S_2, \dots, S_k\}$ and are required to determine whether there is a subset $T \subseteq F$ of disjoint sets such that

$$\bigcup_{S_i \in T} S_i = \bigcup_{S_i \in F} S_i = \{u_1, u_2, \dots, u_n\}$$

From any given instance of this problem, construct the sum of subsets problem $A = \{a_1, \dots, a_k\}$ with $a_j = \sum_{1 \leq i \leq n} \epsilon_{ji}(k+1)^{i-1}$, where $\epsilon_{ji} = 1$ if $u_i \in S_j$ and $\epsilon_{ji} = 0$ otherwise, and $M = \sum_{0 \leq i < n} (k+1)^i = ((k+1)^n - 1)/k$. Clearly, F has an exact cover if and only if $A = \{a_1, \dots, a_k\}$ has a subset with sum M . Since A and M can be constructed from F in polynomial time, exact cover $\propto$ sum of subsets. $\square$

Theorem 11.9 Sum of subsets $\propto$ partition.

Proof: Let $A = \{a_1, \dots, a_n\}$ and M define an instance of the sum of subsets problem. Construct the set $B = \{b_1, b_2, \dots, b_{n+2}\}$ with $b_i = a_i$, $1 \leq i \leq n$, $b_{n+1} = M + 1$, and $b_{n+2} = (\sum_{1 \leq i \leq n} a_i) + 1 - M$. B has a partition if and only if A has a subset with sum M . Since B can be obtained from A and M in polynomial time, sum of subsets $\propto$ partition. $\square$

One can easily show partition $\propto$ 0/1-knapsack and partition $\propto$ job sequencing with deadlines. Hence, these problems are also $\mathcal{NP}$ -hard.

11.4.1 Scheduling Identical Processors

Let P_i , $1 \leq i \leq m$, be m identical processors (or machines). The P_i could, for example, be line printers in a computer output room. Let J_i , $1 \leq i \leq n$, be n jobs. Job J_i requires t_i processing time. A schedule S is an assignment of jobs to processors. For each job J_i , S specifies the time intervals and the processor(s) on which this job is to be processed. A job cannot be processed by more than one processor at any given time. Let f_i be the time at which the processing of job J_i is completed. The *mean finish time* (MFT) of schedule S is

$$\text{MFT}(S) = \frac{1}{n} \sum_{1 \leq i \leq n} f_i$$

Let w_i be a weight associated with each job J_i . The *weighted mean finish time* (WMFT) of schedule S is

$$\text{WMFT}(S) = \frac{1}{n} \sum_{1 \leq i \leq n} w_i f_i$$

Let T_i be the time at which P_i finishes processing all jobs (or job segments) assigned to it. The *finish time* (FT) of S is

$$\text{FT}(S) = \max_{1 \leq i \leq m} \{T_i\}$$

Schedule S is a *nonpreemptive schedule* if and only if each job J_i is processed continuously from start to finish on the same processor. In a *preemptive* schedule each job need not be processed continuously to completion on one processor.

At this point it is worth noting the similarity between the optimal tape storage problem of Section 4.6 and nonpreemptive schedules. Mean retrieval time, weighted mean retrieval time, and maximum retrieval time respectively correspond to mean finish time, weighted mean finish time, and finish time. Minimum finish time schedules can therefore be obtained using the algorithm developed in Section 4.6. Obtaining minimum weighted mean finish time and minimum finish time nonpreemptive schedules is $\mathcal{NP}$ -hard.

Theorem 11.10 Partition $\propto$ minimum finish time nonpreemptive schedule.

Proof: We prove this for $m = 2$. The extension to $m > 2$ is trivial. Let a_i , $1 \leq i \leq n$, be an instance of the partition problem. Define n jobs with processing requirements $t_i = a_i$, $1 \leq i \leq n$. There is a nonpreemptive schedule for this set of jobs on two processors with finish time at most $\sum t_i/2$ iff there is a partition of the a_i 's. $\square$

Example 11.19 Consider the following input to the partition problem: $a_1 = 2, a_2 = 5, a_3 = 6, a_4 = 7$, and $a_5 = 10$. The corresponding minimum finish time nonpreemptive schedule problem has the input $t_1 = 2, t_2 = 5, t_3 = 6, t_4 = 7$, and $t_5 = 10$. There is a nonpreemptive schedule for this set of jobs with finish time 15: P_1 takes the jobs t_2 and t_5 ; P_2 takes the jobs t_1, t_3 , and t_4 . This solution yields a solution for the partition problem also: $\{a_2, a_5\}, \{a_1, a_3, a_4\}$. $\square$

Theorem 11.11 Partition $\propto$ minimum WMFT nonpreemptive schedule.

Proof: Once again we prove this for $m = 2$ only. The extension to $m > 2$ is trivial. Let a_i , $1 \leq i \leq n$, define an instance of the partition problem. Construct a two-processor scheduling problem with n jobs and $w_i = t_i = a_i$, $1 \leq i \leq n$. For this set of jobs there is a nonpreemptive schedule S with weighted mean flow time at most $1/2 \sum a_i^2 + 1/4(\sum a_i)^2$ if and only if the a_i 's have a partition. To see this, let the weights and times of jobs on P_1 be $(\bar{w}_1, \bar{t}_1), \dots, (\bar{w}_k, \bar{t}_k)$ and on P_2 be $(\bar{w}_1, \bar{t}_1), \dots, (\bar{w}_l, \bar{t}_l)$. Assume this is the

order in which the jobs are processed on their respective processors. Then, for this schedule S we have

$$\begin{aligned} n * \text{WMFT}(S) &= \bar{w}_1 \bar{t}_1 + \bar{w}_2(\bar{t}_1 + \bar{t}_2) + \cdots + \bar{w}_k(\bar{t}_1 + \cdots \bar{t}_k) \\ &\quad + \bar{\bar{w}}_1 \bar{\bar{t}}_1 + \bar{\bar{w}}_2(\bar{\bar{t}}_1 + \bar{\bar{t}}_2) + \cdots + \bar{\bar{w}}_l(\bar{\bar{t}}_1 + \cdots \bar{\bar{t}}_l) \\ &= \frac{1}{2} \sum w_i^2 + \frac{1}{2} (\sum \bar{w}_i)^2 + \frac{1}{2} (\sum w_i - \sum \bar{w}_i)^2 \end{aligned}$$

Thus, $n * \text{WMFT}(S) \geq (1/2) \sum w_i^2 + (1/4)(\sum w_i)^2$. This value is obtainable iff the w_i 's (and so also the a_i 's) have a partition. $\square$

Example 11.20 Consider again the partition problem $a_1 = 2, a_2 = 5, a_3 = 6, a_4 = 7$, and $a_5 = 10$. Here, $\frac{1}{2} \sum a_i^2 = \frac{1}{2}(2^2 + 5^2 + 6^2 + 7^2 + 10^2) = 107$, $\sum a_i = 30$, and $\frac{1}{4}(\sum a_i)^2 = 225$. Thus, $1/2 \sum a_i^2 + 1/4(\sum a_i)^2 = 107 + 225 = 332$. The corresponding minimum WMFT nonpreemptive schedule problem has the input $w_i = t_i = a_i$ for $1 \leq i \leq 5$. If we assign the jobs t_2 and t_5 to P_1 and the remaining jobs to P_2 ,

$$n * \text{WFMT}(S) = 5 * 5 + 10(5 + 10) + 2 * 2 + 6(2 + 6) + 7(2 + 6 + 7) = 332$$

The same also yields a solution to the partition problem. $\square$

11.4.2 Flow Shop Scheduling

We shall use the flow shop terminology developed in Section 5.10. When $m = 2$, minimum finish time schedules can be obtained in $O(n \log n)$ time if n jobs are to be scheduled. When $m = 3$, obtaining minimum finish time schedules (whether preemptive or nonpreemptive) is $\mathcal{NP}$ -hard. For the case of nonpreemptive schedules this is easy to see (Exercise 2). We prove the result for preemptive schedules. The proof we give is also valid for the nonpreemptive case. However, a much simpler proof exists for the nonpreemptive case.

Theorem 11.12 Partition $\propto$ the minimum finish time preemptive flow shop schedule ($m > 2$).

Proof: We use only three processors. Let $A = \{a_1, a_2, \dots, a_n\}$ define an instance of the partition problem. Construct the following preemptive flow shop instance FS, with $n + 2$ jobs, $m = 3$ machines, and at most 2 nonzero tasks per job:

$$t_{1,i} = a_i; \quad t_{2,i} = 0; \quad t_{3,i} = a_i, 1 \leq i \leq n$$

$$t_{1,n+1} = T/2; \quad t_{2,n+1} = T; \quad t_{3,n+1} = 0$$

$$t_{1,n+2} = 0; \quad t_{2,n+2} = T; \quad t_{3,n+2} = T/2$$

$$\text{where } T = \sum_{i=1}^n a_i$$

We now show that the preceding flow shop instance has a preemptive schedule with finish time at most $2T$ if and only if A has a partition.

1. If A has a partition u , then there is a nonpreemptive schedule with finish time $2T$. One such schedule is shown in Figure 11.17.
2. If A has no partition, then all preemptive schedules for FS must have a finish time greater than $2T$. This can be shown by contradiction. Assume that there is preemptive schedule for FS with finish time at most $2T$. We make the following observations regarding this schedule:
 - (a) Task $t_{1,n+1}$ must finish by time T as $t_{2,n+1} = T$ and cannot start until $t_{1,n+1}$ finishes.
 - (b) Task $t_{3,n+2}$ cannot start before T units of time have elapsed as $t_{2,n+2} = T$.

Observation (a) implies that only $T/2$ of the first T time units are free on processor one. Let V be the set of indices of tasks completed on processor 1 by time T (excluding task $t_{1,n+1}$). Then,

$$\sum_{i \in V} t_{1,i} < T/2$$

as A has no partition. Hence

$$\sum_{\substack{i \notin V \\ 1 \leq i \leq n}} t_{3,i} > T/2$$

The processing of jobs not included in V cannot commence on processor 3 until after time T since their processor 1 processing is not completed until after T . This together with observation (b) implies that the total amount of processing left for processor 3 at time T is

$$t_{3,n+2} + \sum_{\substack{i \notin V \\ 1 \leq i \leq n}} t_{3,i} > T$$

The schedule length must therefore be more than $2T$. □

$\{t_{1,i} \mid i \in u\}$	$t_{1,n+1}$	$\{t_{1,i} \mid i \notin u\}$	
$t_{2,n+2}$		$t_{2,n+1}$	
	$\{t_{3,i} \mid i \in u\}$	$t_{3,n+2}$	$\{t_{3,i} \mid i \notin u\}$
0	$T/2$	T	$3T/2$
			$2T$

Figure 11.17 A possible schedule

11.4.3 Job Shop Scheduling

A job shop, like a flow shop, has m different processors. The n jobs to be scheduled require the completion of several tasks. The time of the j th task for job J_i is $t_{k,i,j}$. Task j is to be performed on processor P_k . The tasks for any job J_i are to be carried out in the order $1, 2, 3, \dots$, and so on. Task j cannot begin until task $j - 1$ (if $j > 1$) has been completed. Note that it is quite possible for a job to have many tasks that are to be performed on the same processor. In a nonpreemptive schedule, a task once begun is processed without interruption until it is completed. The definitions of $\text{FT}(S)$ and $\text{MFT}(S)$ extend to this problem in a natural way. Obtaining either a minimum finish time preemptive schedule or a minimum finish time nonpreemptive schedule is $\mathcal{NP}$ -hard even when $m = 2$. The proof for the nonpreemptive case is very simple (use partition). We present the proof for the preemptive case. This proof will also be valid for the nonpreemptive case but will not be the simplest proof for this case.

Theorem 11.13 Partition $\propto$ minimum finish time preemptive job shop schedule ($m > 1$).

Proof: We use only two processors. Let $A = \{a_1, a_2, \dots, a_n\}$ define an instance of the partition problem. Construct the following job shop instance JS, with $n + 1$ jobs and $m = 2$ processors.

$$\begin{aligned} \text{Jobs } 1, \dots, n: & \quad t_{1,i,1} = t_{2,i,2} = a_i \quad \text{for } 1 \leq i \leq n \\ \text{Job } n + 1: & \quad t_{2,n+1,1} = t_{1,n+1,2} = t_{2,n+1,3} = t_{1,n+1,4} = T/2 \end{aligned}$$

$$\text{where } T = \sum_{i=1}^n a_i$$

We show that the job shop problem has a preemptive schedule with finish time at most $2T$ if and only if S has a partition.

1. If A has a partition, u then there is a schedule with finish time $2T$ (see Figure 11.18).
2. If A has no partition, then all schedules for JS must have a finish time greater than $2T$. To see this, assume that there is a schedule S for JS with finish time at most $2T$. Then, job $n+1$ must be scheduled as in Figure 11.18. Also, there can be no idle time on either P_1 or P_2 . Let R be the set of jobs scheduled on P_1 in the interval $[0, T/2]$. Let R' be the subset of R representing jobs whose first task is completed on P_1 in this interval. Since the a_i 's have no partition, $\sum_{j \in R'} t_{i,j,1} < T/2$. Consequently, $\sum_{j \in R'} t_{2,j,2} < T/2$. Since only the second tasks of jobs in R' can be scheduled on P_2 in the interval $[T/2, T]$, it follows that there is some idle time on P_2 in this interval. Hence, S must have finish time greater than $2T$. $\square$

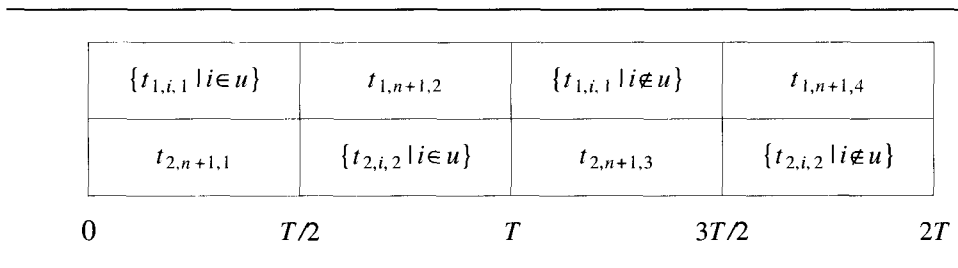


Figure 11.18 Another schedule

EXERCISES

1. [Job sequencing] Show that the job sequencing with deadlines problem (Section 8.1.4) is $\mathcal{NP}$ -hard.
2. Show that $\text{partition} \propto$ the minimum finish time nonpreemptive three-processor flow shop schedule. Use only one job that has three nonzero tasks. All other jobs have only one nonzero task.
3. Show that $\text{partition} \propto$ the minimum finish time nonpreemptive two-processor job shop schedule. Use only one job that has three nonzero tasks. All other jobs have only one nonzero task.
4. Let $J_1, \dots, J_n$ be n jobs. Job i has a processing time t_i and a deadline d_i . Job i is not available for processing until time r_i . Show that deciding whether all n jobs can be processed on one machine without violating any deadline is $\mathcal{NP}$ -hard. (*Hint:* Use partition.)

5. Let J_i , $1 \leq i \leq n$, be n jobs as in Exercise 4. Assume $r_i = 0$, $1 \leq i \leq n$. Let f_i be the finish time of J_i in a one-processor schedule S . The *tardiness* T_i of J_i is $\max\{0, f_i - d_i\}$. Let w_i , $1 \leq i \leq n$, be nonnegative weights associated with the J_i 's. The total weighted tardiness is $\sum w_i T_i$. Show that finding a schedule minimizing $\sum w_i T_i$ is $\mathcal{NP}$ -hard. (*Hint*: Use partition).
6. Let J_i , $1 \leq i \leq n$, be n jobs. Job J_i has a processing time of t_i . Its processing cannot begin until time r_i . Let w_i be a weight associated with J_i . Let f_i be the finish time of J_i in a one-processor schedule S . Show that finding a one-processor schedule that minimizes $\sum w_i f_i$ is $\mathcal{NP}$ -hard.
7. Show that the problem of obtaining optimal finish time preemptive schedules for a two-processor flow shop is $\mathcal{NP}$ -hard when jobs are released at two different times R_1 and R_2 . Jobs released at R_i cannot be scheduled before R_i .

11.5 $\mathcal{NP}$ -HARD CODE GENERATION PROBLEMS

The function of a compiler is to translate programs written in some source language into an equivalent assembly language or machine language program. Thus, the C++ compiler on the Sparc 10 translates C++ programs into the machine language of this machine. We look at the problem of translating arithmetic expressions in a language such as C++ into assembly language code. The translation clearly depends on the particular assembly language (and hence machine) being used. To begin, we assume a very simple machine model. We call this model machine A . This machine has only one register called the *accumulator*. All arithmetic has to be performed in this register. If $\odot$ represents a binary operator such as $+$, $-$, $*$, and $/$, then the left operand of $\odot$ must be in the accumulator. For simplicity, we restrict ourselves to these four operators. The discussion easily generalizes to other operators. The relevant assembly language instructions are:

LOAD X load accumulator with contents of memory location X .
 STORE X store contents of accumulator into memory location X .
 OP X OP may be ADD, SUB, MPY, or DIV.

The instruction OP X computes the operator OP using the contents of the accumulator as the left operand and that of memory location X as the right operand. As an example, consider the arithmetic expression $(a+b)/(c+d)$. Two possible assembly language versions of this expression are given in Figure 11.19. $T1$ and $T2$ are temporary storage areas in memory. In both

cases the result is left in the accumulator. Code (a) is two instructions longer than code (b). If each instruction takes the same amount of time, then code (b) will take 25% less time than code (a). For the expressions $(a + b)/(c + d)$ and the given machine A , it is easy to see that code (b) is optimal.

LOAD	a	LOAD	c
ADD	b	ADD	d
STORE	$T1$	STORE	$T1$
LOAD	c	LOAD	a
ADD	d	ADD	b
STORE	$T2$	DIV	$T1$
LOAD	$T1$		
DIV	$T2$		

(a)
(b)

Figure 11.19 Two possible codes for $(a + b)/(c + d)$

Definition 11.7 A *translation* of an expression E into the machine or assembly language of a given machine is *optimal* if and only if it has a minimum number of instructions. □

Definition 11.8 A binary operator $\odot$ is *commutative* in the domain D iff $a \odot b = b \odot a$ for all a and b in D . □

Machine A can be generalized to another machine B . Machine B has $N \geq 1$ registers in which arithmetic can be performed. There are four types of machine instructions for B :

1. LOAD M, R
2. STORE M, R
3. OP $R1, M, R2$
4. OP $R1, R2, R3$

These four instruction types perform the following functions:

1. LOAD M, R places the contents of memory location M into register $R, 1 \leq R \leq N$.
2. STORE M, R stores the contents of register $R, 1 \leq R \leq N$, into memory location M .

3. OP $R1, M, R2$ computes $\text{contents}(R1) \text{ OP } \text{contents}(M)$ and places the result in register $R2$. OP is any binary operator (for example, $+$, $-$, $*$, or $/$); $R1$ and $R2$ are registers; $R1$ may equal $R2$; M is a memory location.
4. OP $R1, R2, R3$ is similar to instruction type (3). Here $R1, R2$, and $R3$ are registers. Some or all of these registers may be the same.

In comparing the two machine models A and B , we note that when $N = 1$, instructions of types (1), (2) and (3) for model B are the same as the corresponding instructions for model A . Instructions of type (4) only allow trivial operations like $a + a$, $a - a$, $a * a$, and a/a to be performed without an additional memory access. This does not change the number of instructions in the optimal codes for A and B when $N = 1$. Hence, model A is in a sense identical to model B when $N = 1$. For model B , we see that the optimal code for a given expression E may be different for different values of N . Figure 11.20 shows the optimal code for the expression $(a + b)/(c * d)$. Two cases are considered, $N = 1$ and $N = 2$. Note that when $N = 1$, one store has to be made whereas when $N = 2$, no stores are needed. The registers are labeled $R1$ and $R2$. Register $T1$ is a temporary storage location in memory.

LOAD	$c, R1$	LOAD	$c, R1$
MPY	$R1, d, R1$	MPY	$R1, d, R1$
STORE	$R1, T1$	LOAD	$a, R2$
LOAD	$a, R1$	ADD	$R2, b, R2$
ADD	$R1, b, R1$	DIV	$R2, R1, R1$
DIV	$R1, T1, R1$		

(a) $N = 1$
(b) $N = 2$

Figure 11.20 Optimal codes for $N = 1$ and $N = 2$

Given an expression E , the first question we ask is: can E be evaluated without any STOREs? A closely related question is: what is the minimum number of registers needed to evaluate E without any stores? We show that this problem is $\mathcal{NP}$ -hard.

11.5.1 Code Generation with Common Subexpressions

When arithmetic expressions have common subexpressions, they can be represented by a directed acyclic graph (dag). Every internal node (node with

nonzero out-degree) in the dag represents an operator. Assuming the expression contains only binary operators, each internal node P has out-degree two. The two nodes adjacent from P are called the left and right children of P respectively. The children of P are the roots of the dags for the left and right operands of P . Node P is the parent of its children. Figure 11.21 shows some expressions and their dag representations.

Definition 11.9 A *leaf* is a node with out-degree zero. A *level-one* node is a node both of whose children are leaves. A *shared node* is a node with more than one parent. A *leaf dag* is a dag in which all shared nodes are leaves. A *level-one dag* is a dag in which all shared nodes are level-one nodes. $\square$

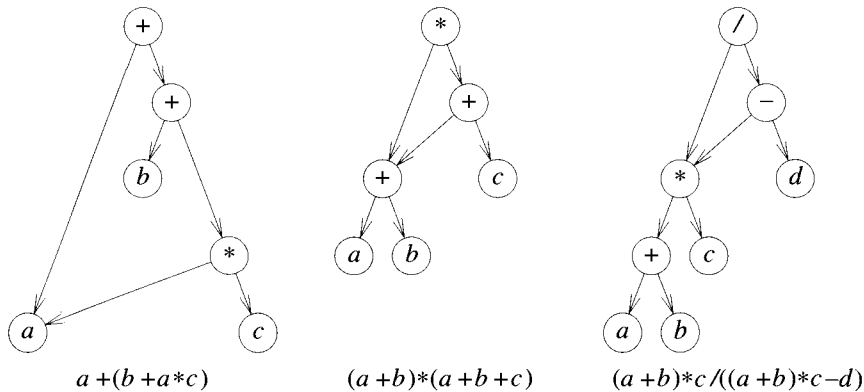


Figure 11.21 Expressions and their dags

Example 11.21 The dag of Figure 11.21(a) is a leaf dag. Figure 11.21(b) is a level-one dag. Figure 11.21(c) is neither a leaf dag nor a level-one dag. $\square$

A leaf dag results from an arithmetic expression in which the only common subexpressions are simple variables or constants. A level-one dag results from an expression in which the only common subexpressions are of the form $a \odot b$, where a and b are simple variables or constants and $\odot$ is an operator.

The problem of generating optimal code for level-one dags is $\mathcal{NP}$ -hard even when the machine for which code is being generated has only one register. Determining the minimum number of registers needed to evaluate a dag with no STOREs is also $\mathcal{NP}$ -hard.

Example 11.22 The optimal codes for the dag of Figure 11.21(b) for one- and two-register machines is given in Figure 11.22.

The minimum number of registers needed to evaluate this dag without any STOREs is two. $\square$

LOAD	a,R1	LOAD	a,R1
ADD	R1,b,R1	ADD	R1,b,R1
STORE	T1,R1	ADD	R1,c,R2
ADD	R1,c,R1	MUL	R1,R2,R1
STORE	T2,R1		
LOAD	T1,R1		
MUL	R1,T2, R1		
(a)		(b)	

Figure 11.22 Optimal codes for one- and two-register machines

To prove the above statements, we use the feedback node set (FNS) problem that is shown to be $\mathcal{NP}$ -hard in Exercise 5 (Section 11.3).

FNS: Given a directed graph $G = (V, E)$ and an integer k , determine whether there exists a subset V' of vertices $V' \subseteq V$ and $|V'| \leq k$ such that the graph $H = (V - V', E - \{\langle u, v \rangle | u \in V' \text{ or } v \in V'\})$ obtained from G by deleting all vertices in V' and all edges incident to a vertex in V' contains no directed cycles.

We explicitly prove only that generating optimal code is $\mathcal{NP}$ -hard. Using the construction of this proof, we can also show that determining the minimum number of registers needed to evaluate a dag with no STOREs is $\mathcal{NP}$ -hard as well. The proof assumes that expressions can contain commutative operators and that shared nodes may be computed only once. It is easily extended to allow recomputation of shared nodes. Using an idea due to R. Sethi, the proof is easily extended to the case in which only noncommutative operations are allowed (see Exercise 1).

Theorem 11.14 $\text{FNS} \propto$ the optimal code generation for level-one dags on a one-register machine.

Proof: Let G, k be an instance of FNS. Let n be the number of vertices in G . We construct a dag A with the property that the optimal code for the expression corresponding to A has at most $n + k$ LOADs if and only if G has a feedback node set of size at most R .

The dag A consists of three kinds of nodes: leaf nodes, chain nodes, and tree nodes. All chain and tree nodes are internal nodes representing commutative operators (for example, $+$). Leaf nodes represent distinct variables. We use d_v to denote the out-degree of vertex v of G . Corresponding to each vertex v of G , there is a directed chain of chain nodes $v_1, v_2, \dots, v_{d_v+1}$ in A . Node v_{d_v+1} is the *head node* of the chain for v and is the parent of two leaf nodes v_L and v_R (see Example 11.23 and Figure 11.23). Vertex v_1 is the *tail* of the chain. From each of the chain nodes corresponding to vertex v , except the head node, there is one directed edge to the head node of one of the chains corresponding to a vertex w such that $\langle v, w \rangle$ is an edge in G . Each such edge goes to a distinct head. Note that as a result of the addition of these edges, each chain node now has out-degree two. Since each chain node represents a commutative operator, it does not matter which of its two children is regarded as the left child.

At this point we have a dag in which the tail of every chain has in-degree zero. We now introduce tree nodes to combine all the tails so that we are left with only one node (the root) with in-degree zero. Since G has n vertices, we need $n - 1$ tree nodes (note that every binary tree with $n - 1$ internal nodes has n external nodes). These $n - 1$ nodes are connected together to form a binary tree (any binary tree with $n - 1$ nodes will do). In place of the external nodes we connect the tails of the n chains (see Figure 11.23(b)). This yields a dag A corresponding to an arithmetic expression.

It is easy to see that every optimal code for A will have exactly n LOADs of leaf nodes. Also, there will be exactly one instruction of type $\odot$ for every chain node and tree node (we assume that a shared node is computed only once). Hence, the only variable is the number of LOADs and STOREs of chain and tree nodes. If G has no directed cycles, then its vertices can be arranged in topological order (vertex u precedes vertex v in a topological ordering only if there is no directed path from u to v in G). Let $v_1, v_2, \dots, v_n$ be a topological ordering of the vertices in G . The expression A can be computed using no LOADs of chain and tree nodes by first computing all nodes on the chain for v_n and storing the result of the tail node. Next, all nodes on the chain for v_{n-1} can be computed. In addition, we can compute any nodes on the path from the tail for v_{n-1} to the root for which both operands are available. Finally, one result needs to be stored. Next, the chain for v_{n-2} can be computed. Again, we can compute all nodes on the path from this chain tail to the root for which both operands are available. Continuing in this way, the entire expression can be computed.

If G contains at least one cycle $v_1, v_2, \dots, v_i, v_1$, then every code for A must contain at least one LOAD of a chain node on a chain for one of $v_1, v_2, \dots, v_i$. Further, if none of these vertices is on any other cycle, then all their chain nodes can be computed using only one load of a chain node. This argument is readily generalized to show that if the size of a minimum feedback node set is p , then every optimal code for A contains exactly $n + p$

LOADs. The p LOADs correspond to a combination of tail nodes corresponding to a minimum feedback node set and the siblings of these tail nodes. If we had used noncommutative operators for chain nodes and made each successor on a chain the left child of its parent, then the p LOADs would correspond to the tails of the chains of any minimum feedback set. Furthermore, if the optimal code contains p LOADs of chain nodes, then G has a feedback node set of size p . $\square$

Example 11.23 Figure 11.23(b) shows the dag A corresponding to the graph G of Figure 11.23(a). The set $\{r, s\}$ is a minimum feedback node set for G . The operator in each chain and tree node can be assumed to be $+$. Each code for A has a load corresponding to one of (p_L, p_R) , (q_L, q_R) , $\dots$, and (u_L, u_R) . The expression A can be computed using only two additional LOADs by computing nodes in the order $r_4, s_2, q_2, q_1, p_2, p_1, c, u_3, u_2, u_1, t_2, t_1, e, s_1, r_3, r_2, r_1, d, b$, and a . Note that a LOAD is needed to compute s_1 and also to compute r_3 . $\square$

11.5.2 Implementing Parallel Assignment Instructions

A *parallel assignment instruction* has the format $(v_1, v_2, \dots, v_n) := (e_1, e_2, \dots, e_n)$ where the v_i 's are distinct variable names and the e_i 's are expressions. The semantics of this statement is that the value of v_i is updated to be the value of the expression e_i , $1 \leq i \leq n$. The value of the expression e_i is to be computed using the values the variables in e_i have before this instruction is executed.

Example 11.24 1. $(A, B) := (B, C)$; is equivalent to $A := B; B := C$;
 2. $(A, B) := (B, A)$; is equivalent to $T := A; A := B; B := T$;
 3. $(A, B) := (A + B, A - B)$; is equivalent to $T1 := A; T2 := B; A := T1 + T2; B := T1 - T2$; and also to $T1 := A; A := A + B; B := T1 - B$;
 $\square$

As the above example indicates, it may be necessary to store some of the v_i 's in temporary locations when executing a parallel assignment. These stores are needed only when some of the v_i 's appear in the expressions e_j , $1 \leq j \leq n$. A variable v_i is *referenced* by expression e_i if and only if v_i appears in e_j . It should be clear that only referenced variables need to be copied into temporary locations. Further, parts (2) and (3) of Example 11.24 show that not all referenced variables need to be copied.

An implementation of a parallel assignment statement is a sequence of instructions of types $T_j = v_i$ and $v_i = e'_i$, where e'_i is obtained from e_i by replacing all occurrences of a v_i that have already been updated with references to the temporary locations in which the old values of v_i has been

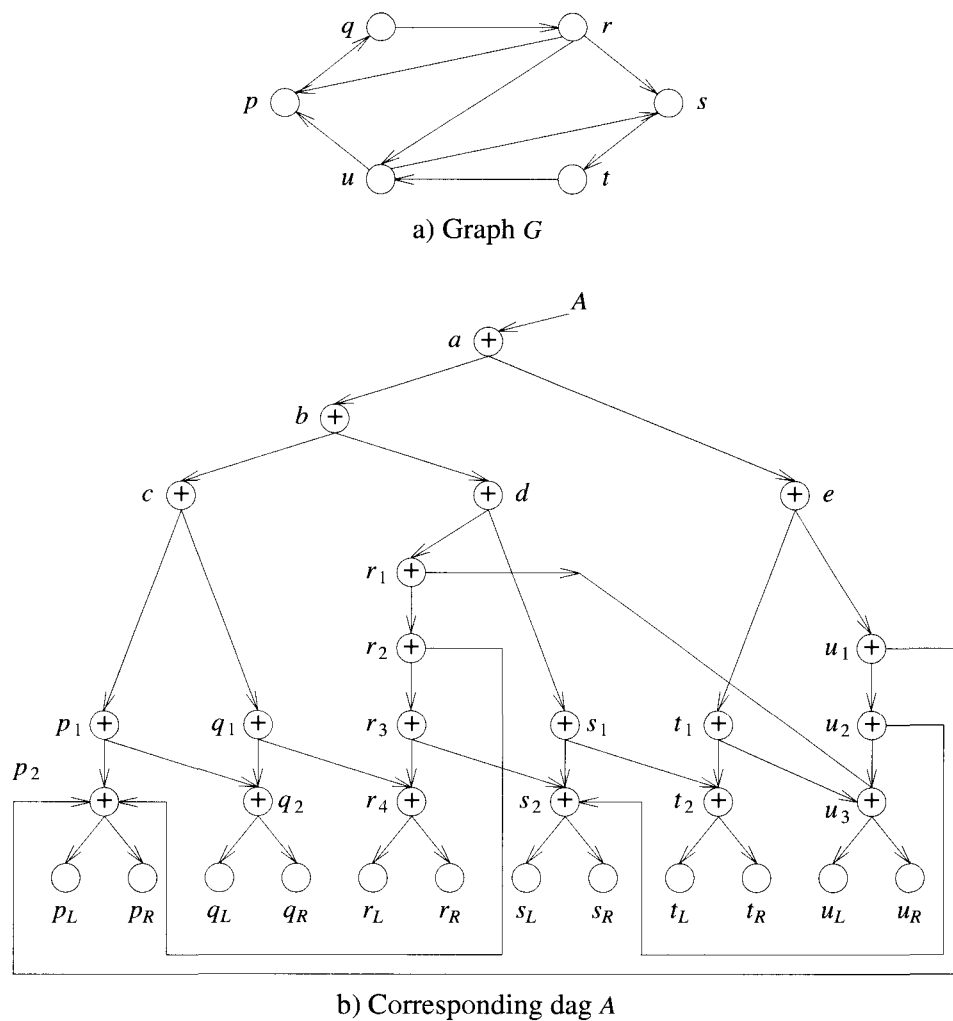


Figure 11.23 A graph and its corresponding dag

saved. Let $R = (\tau(1), \dots, \tau(n))$ be a permutation of $(1, 2, \dots, n)$. Then R is a *realization* of an assignment statement. It specifies the order in which statements of type $v_i = e'_i$ appear in an implementation of a parallel assignment statement. The order is $v_{\tau(1)} = e'_{\tau(1)}, v_{\tau(2)} = e'_{\tau(2)}$, and so on. The implementation also has statements of type $T_j = v_i$ interspersed. Without loss of generality we can assume that the statement $T_j = v_i$ (if it appears in the implementation) immediately precedes the statement $v_i = e'_i$. Hence, a realization completely characterizes an implementation. The minimum number of instructions of type $T_j = v_i$ for any given realization is easy to determine. This number is the cost of the realization. The *cost* $C(R)$ of a realization R is the number of v_i that are referenced by an e_j that corresponds to an instruction $v_j = e'_j$ that appears after the instruction $v_i = e'_i$.

Example 11.25 Consider the statement $(A, B, C) := (D, A + B, A - B);$. The $3! = 6$ different realizations and their costs are given in Figure 11.24. The realization 3, 2, 1 corresponding to the implementation $C = A - B; B = A + B; A = D;$ needs no temporary stores ($C(R) = 0$). $\square$

R	$C(R)$
1, 2, 3	2
1, 3, 2	2
2, 1, 3	2
2, 3, 1	1
3, 1, 2	1
3, 2, 1	0

Figure 11.24 Realization for Example 11.25

An optimal realization for a parallel assignment statement is one with minimum cost. When the expressions e_i are all variable names or constants, an optimal realization can be found in linear time ($O(n)$). When the e_i are allowed to be expressions with operators then finding an optimal realization is $\mathcal{NP}$ -Hard. We prove this statement using the feedback node set problem.

Theorem 11.15 FNS $\propto$ the minimum-cost realization.

Proof: Let $G = (V, E)$ be any n -vertex directed graph. Construct the parallel assignment statement $P : (v_1, v_2, \dots, v_n) := (e_1, e_2, \dots, e_n)$, where the v_i 's correspond to the n vertices in V and e_i is the expression $v_{i_1} + v_{i_2} + \dots + v_{i_j}$. The set $\{v_{i_1}, v_{i_2}, \dots, v_{i_j}\}$ is the set of vertices adjacent from v_i

(that is, $\langle v_i, v_{i_j} \rangle \in E(G)$, $1 \leq i \leq j$). This construction requires at most $O(n^2)$ time.

Let U be any feedback node set for G . Let $G' = (V', E') = (V - U, E - \{\langle x, y \rangle | x \in U \text{ or } y \in U\})$ be the graph obtained by deleting vertex set U and all edges incident to vertices in U . From the definition of a feedback node set, it follows that G' is acyclic. So, the vertices in $V - U$ can be arranged in a sequence $s_1, s_2, \dots, s_m$, where $m = |V - U|$ and E' contains no edge $\langle s_j, s_i \rangle$ for any i and j , $1 \leq i < j \leq m$. Hence, an implementation of P in which variables corresponding to vertices in U are first stored in temporary locations followed by the instructions $v_i = e'_i$ corresponding to $v_i \in U$, followed by the corresponding instructions for $s_1, s_2, \dots, s_m$ (in that order), will be a correct implementation. (Note that e'_i is e_i with all occurrences of $v_i \in U$ replaced by the corresponding temporary location.) The realization R corresponding to this implementation has $C(R) = |U|$. Hence, if G has a feedback node set of size at most k , then P has an optimal realization of cost at most k .

Suppose P has a realization R of cost k . Let U be the set of k variables that have to be stored in temporary locations and let $R = (q_1, q_2, \dots, q_n)$. From the definition of $C(R)$ it follows that no e_{q_i} references a v_{q_j} with $j < i$ unless $v_{q_j} \in U$. Hence, the deletion of vertices in U from G leaves G acyclic. Thus, U defines a feedback node set of size k for G .

G has a feedback node set of size at most k if and only if P has a realization of cost at most k . Thus we can solve the feedback node set problem in polynomial time if we have a polynomial time algorithm that determines a minimum-cost realization. $\square$

EXERCISES

1. (a) How should the proof of Theorem 11.14 be modified to permit recomputation of shared nodes?
- (b) [R. Sethi] Modify the proof of Theorem 11.14 so that it holds for level-one dags representing expressions in which all operators are noncommutative. (*Hint*: Designate the successor vertex on a chain to be the left child of its predecessor vertex and use the $n+1$ node binary tree of Figure 11.25 to connect together the tail nodes of the n chains.)
- (c) Show that optimal code generation is $\mathcal{NP}$ -hard for leaf dags on an infinite register machine. (*Hint*: Use FNS.)

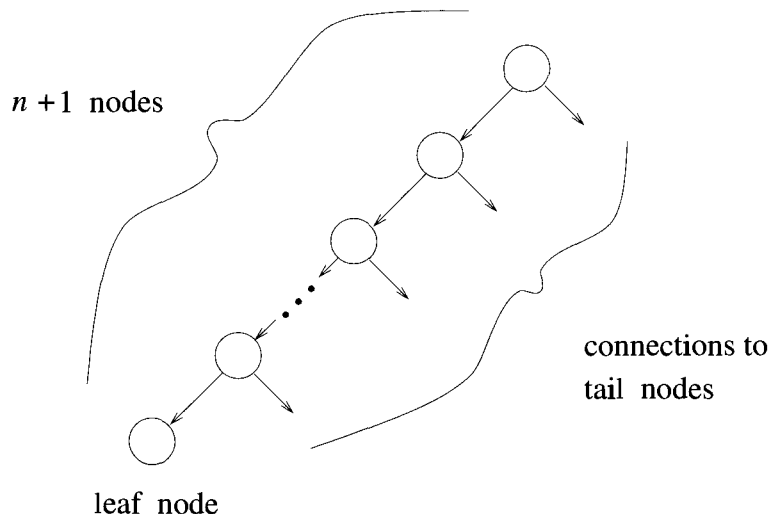


Figure 11.25 Figure for Exercise 1

11.6 SOME SIMPLIFIED $\mathcal{NP}$ -HARD PROBLEMS

Once we have shown a problem L to be $\mathcal{NP}$ -hard, we would be inclined to dismiss the possibility that L can be solved in deterministic polynomial time. At this point, however, we can naturally ask the question: Can a suitably restricted version (i.e., some subclass) of an $\mathcal{NP}$ -hard problem be solved in deterministic polynomial time? It should be easy to see that by placing enough restrictions on any $\mathcal{NP}$ -hard problem (or by defining a sufficiently small subclass), we can arrive at a polynomially solvable problem. As examples, consider the following:

1. CNF-satisfiability with at most three literals per clause is $\mathcal{NP}$ -hard. If each clause is restricted to have at most two literals, then CNF-satisfiability is polynomially solvable.
2. Generating optimal code for a parallel assignment statement is $\mathcal{NP}$ -hard. However, if the expressions e_i are restricted to be simple variables, then optimal code can be generated in polynomial time.
3. Generating optimal code for level-one dags is $\mathcal{NP}$ -hard, but optimal code for trees can be generated in polynomial time.

4. Determining whether a planar graph is three colorable is $\mathcal{NP}$ -hard. To determine whether it is two colorable, we only have to see whether it is bipartite.

Since it is very unlikely that $\mathcal{NP}$ -hard problems are polynomially solvable, it is important to determine the weakest restrictions under which we can solve a problem in polynomial time.

To narrow the gap between subclasses for which polynomial time algorithms are known and those for which such algorithms are not known, it is desirable to obtain as strong a set of restrictions under which a problem remains $\mathcal{NP}$ -hard or $\mathcal{NP}$ -complete.

We state without proof the severest restrictions under which certain problems are known to be $\mathcal{NP}$ -hard or $\mathcal{NP}$ -complete. We state these simplified or restricted problems as decision problems. For each problem we specify only the input and the decision to be made.

Theorem 11.16 The following decision problems are $\mathcal{NP}$ -complete.

1. **Node cover**

Input: An undirected graph G with node degree at most 3 and an integer k .

Decision: Does G have a node cover of size at most k ?

2. **Planar Node Cover**

Input: A planar undirected graph G with node degree at most 6 and an integer k .

Decision: Does G have a node cover of size at most k ?

3. **Colorability**

Input: A planar undirected graph G with node degree at most four.

Decision: Is G three colorable?

4. **Undirected Hamiltonian Cycle**

Input: An undirected graph G with node degree at most three.

Decision: Does G have a Hamiltonian cycle?

5. **Planar Undirected Hamiltonian Cycle**

Input: A planar undirected graph.

Decision: Does G have a Hamiltonian cycle?

6. **Planar Directed Hamiltonian Path**

Input: A planar directed graph G with in-degree at most 3 and out-degree at most 4.

Decision: Does G have a directed Hamiltonian path?

7. **Unary Input Partition**

Input: Positive integers a_i , $1 \leq i \leq m$, n , and B such that

$$\sum_{1 \leq i \leq m} a_i = nB, \quad \frac{B}{4} < a_i < \frac{B}{2}, \quad 1 \leq i \leq m, \quad m = 3n$$

Input is in unary notation.

Decision: Is there a partition $\{A_1, \dots, A_n\}$ of the a_i 's such that each A_i contains three elements and

$$\sum_{a \in A_i} a = B, \quad 1 \leq i \leq n$$

8. **Unary Flow Show**

Input: Task times in unary notation and an integer T .

Decision: Is there a two-processor nonpreemptive schedule with mean finish time at most T ?

9. **Simple Max Cut**

Input: A graph $G = (V, E)$ and an integer k .

Decision: Does V have a subset V_1 such that there are at least k edges $(u, v) \in E$ with $u \in V_1$ and $v \notin V_1$?

10. **SAT2**

Input: A propositional formula F in CNF. Each clause in F has at most two literals. An integer k .

Decision: Can at least k clauses of F be satisfied?

11. **Minimum Edge Deletion Bipartite Subgraph**

Input: An undirected graph G and an integer k .

Decision: Can G be made bipartite by the deletion of at most k edges?

12. **Minimum Node Deletion Bipartite Subgraph**

Input: An undirected graph G and an integer k .

Decision: Can G be made bipartite by the deletion of at most k vertices?

13. **Minimum Cut into Equal-Sized Subsets**

Input: An undirected graph $G = (V, E)$, two distinguished vertices s and t , and a positive integer W .

Decision: Is there a partition $V = V_1 \cup V_2$, $V_1 \cap V_2 = \phi$, $|V_1| = |V_2|$, $s \in V_1$, $t \in V_2$, and $|\{(u, v) | u \in V_1, v \in V_2 \text{ and } (u, v) \in E\}| \leq W$?

14. Simple Optimal Linear Arrangement

Input: An undirected graph $G = (V, E)$ and an integer k . $|V| = n$.

Decision: Is there a one-to-one function $f : V \rightarrow \{1, 2, \dots, n\}$ such that

$$\sum_{(u,v) \in E} |f(u) - f(v)| \leq k$$

11.7 REFERENCES AND READINGS

A comprehensive treatment of $\mathcal{NP}$ -hard and $\mathcal{NP}$ -complete problems can be found in *Computers and intractability: A Guide to the Theory of NP-Completeness*, by M. Garey and D. Johnson, W. H. Freeman, 1979.

Our proof satisfiability $\propto$ directed Hamiltonian cycle is due to P. Hermann. The proof satisfiability $\propto$ AND/OR Graphs and the proof used in the text for Theorem 11.11 were given by S. Sahni. Theorem 11.11 is due to J. Bruno, E. G. Coffman, Jr., and R. Sethi.

Theorems 11.12 and 11.13 are due to T. Gonzalez and S. Sahni. The proof of Theorem 11.13 is due to D. Nassimi. The proof of Theorem 11.14 is due to A. Aho, S. Johnson, and J. Ullman.

The fact that the code generation problem for one-register machines is $\mathcal{NP}$ -hard was first proved by J. Bruno and R. Sethi. The result in their paper is stronger than Theorem 11.14 as it applies even to expressions containing no commutative operators. Theorem 11.15 is due to R. Sethi.

The results stated in Section 11.6 were presented by D. Johnson and L. Stockmeyer.

For additional material on complexity theory see *Complexity Theory*, by C. H. Papadimitriou, Addison-Wesley, 1994.

11.8 ADDITIONAL EXERCISES

1. [Circuit realization] Let C be a circuit made up of **and**, **or**, and **not** gates. Let $x_1, \dots, x_n$ be the inputs and f the output. Show that deciding whether $f(x_1, \dots, x_n) = F(x_1, \dots, x_n)$, where F is a propositional formula, is $\mathcal{NP}$ -hard.
2. Show that determining whether C is a minimum circuit (i.e., has a minimum number of gates, see Exercise 1) realizing a formula F is $\mathcal{NP}$ -hard.

3. [0/1 knapsack] Show that Partition $\propto$ the 0/1 knapsack decision problem.
4. [Quadratic programming] Show that finding the maximum of a function $f(x_1, \dots, x_n)$ subject to the linear constraints $\sum_{1 \leq i < j \leq n} a_{ij}x_i x_j \leq b_i$, $1 \leq i \leq n$, and $x_i \geq 0$, $1 \leq i \leq n$ is $\mathcal{NP}$ -hard. The function f is restricted to be of the form $\sum c_i x_i^2 + \sum d_i x_i$.
5. Let $G = (V, E)$ be a graph. Let $w(i, j)$ be a weighting function for the edges of G . A *cut* of G is a subset $S \subseteq V$. The *weight* of a cut is

$$\sum_{i \in S, j \notin S} w(i, j)$$

A *max-cut* is a cut of maximum weight. Show that the problem of determining the weight of a max-cut is $\mathcal{NP}$ -hard.

6. [Plant location] Let S_i , $1 \leq i \leq n$, be n possible sites at which plants can be located. At each site at most one plant can be located. If a plant is located at site S_i , then a fixed cost F_i is incurred. This is the cost of setting up the plant. A plant located at S_i has a maximum production capacity of C_i . There are n destinations D_i , $1 \leq i \leq m$, to which products have to be shipped. The demand at D_i is d_i , $1 \leq i \leq m$. The per-unit cost of shipping a product from site i to destination j is c_{ij} . A destination can be supplied from many plants. Define $y_i = 0$ if no plant is located at site i and $y_i = 1$ otherwise. Let x_{ij} be the number of units of the product shipped from S_i to D_j . Then, the total cost is

$$\sum_i F_i y_i + \sum_i \sum_j c_{ij} x_{ij}, \quad \sum_i x_{ij} = d_j, \quad \text{and} \quad \sum_j x_{ij} \leq C_i y_i$$

All x_{ij} are nonnegative integers. We assume that $\sum C_{ij} \geq \sum d_i$. Show that finding y_i and x_{ij} so that the total cost is minimized is $\mathcal{NP}$ -hard.

7. [Concentrator location] This problem is very similar to the plant location problem (Exercise 6). The only difference is that each destination may be supplied by only one plant. When this restriction is imposed, the plant location problem becomes the concentrator location problem arising in computer network design. The destinations represent computer terminals. The plants represent the concentration of information from the terminals which they supply. Show that the concentrator location problem is $\mathcal{NP}$ -hard under each of the following conditions:

- (a) $n = 2$, $C_1 = C_2$, and $F_1 = F_2$. (*Hint*: Use Partition.)
- (b) $F_i/C_i = F_{i+1}/C_{i+1}$, $1 \leq i < n$, and $d_i = 1$. (*Hint*: Use exact cover.)

8. [Steiner trees] Let T be a tree and R a subset of the vertices in T . Let $w(i, j)$ be the weight of edge (i, j) in T . If (i, j) is not an edge in T , then $w(i, j) = \infty$. A Steiner tree is a subtree of T that includes the vertex set R . It may include other vertices too. Its cost is the sum of the weights of the edges in it. Show that finding a minimum-cost Steiner tree is $\mathcal{NP}$ -hard.
9. Assume that P is a parallel assignment statement $(v_1, \dots, v_n) := (e_1, \dots, e_n)$, where each e_i is a simple variable and the v_i 's are distinct. For convenience, assume that the distinct variables in P are $(v_1, \dots, v_m)$ with $m \geq n$ and that $E = (i_1, i_2, \dots, i_n)$ is a set of indices such that $e_{i_j} = v_{i_j}$. Then write an $O(n)$ time algorithm to find an optimal realization for P .
10. Let $F = \{S_j\}$ be a finite family of sets. Let $T \leq F$ be a subfamily of F . The size of T , $|T|$, is the number of sets in T . Let S_i and S_j be two sets in T . Also S_i and S_j are disjoint if and only if $S_i \cap S_j = \phi$. T is a disjoint subset of F if and only if every two sets in T are disjoint. The set packing problem is to determine a disjoint subfamily T of maximum size. Show that clique $\propto$ set packing.
11. Show that the following decision problem is $\mathcal{NP}$ -complete.
Input: Positive integer n ; w_i , $1 \leq i \leq n$, and M .
Decision: Do there exist nonnegative integers $x_i \geq 0$, $1 \leq i \leq n$; such that

$$\sum_{1 \leq i \leq n} w_i x_i = M$$
12. An *independent set* in an undirected graph $G(V, E)$ is a set of vertices no two of which are connected. Given a graph G and an integer k , the problem is to determine whether G has an independent set of size k . Show that this problem is $\mathcal{NP}$ -complete.
13. Given an undirected graph $G(V, E)$ and an integer k , the goal is to determine whether G has a clique of size k **and** an independent set of size k . Show that this problem is $\mathcal{NP}$ -complete.
14. Is the following problem in $\mathcal{P}$? If yes, give a polynomial time algorithm; if not, show it is $\mathcal{NP}$ -complete.

Input are an undirected graph $G = (V, E)$ of degree 1000 and an integer $k(\leq |V|)$. Decide whether G has a clique of size k .

15. Given an integer $m \times n$ matrix A and an integer $m \times 1$ vector b , the *0-1 integer programming problem* asks whether there is an integer $n \times 1$ vector x with elements in the set $\{0, 1\}$ such that $Ax \leq b$. Prove that 0-1 integer programming is $\mathcal{NP}$ -complete.

16. Input are finite sets $A_1, A_2, \dots, A_m$ and $B_1, B_2, \dots, B_n$. The *set intersection problem* is to decide whether there is a set T such that $|T \cap A_i| \geq 1$ for $i = 1, 2, \dots, m$, and $|T \cap B_j| \leq 1$ for $j = 1, 2, \dots, n$. Show that the set intersection problem is $\mathcal{NP}$ -complete.
17. We say an undirected graph $G(V, E)$ is k colorable if each node of G can be labeled with an integer in the range $[1, k]$, such that no two nodes connected by an edge have the same label. Is the following problem in $\mathcal{P}$? If yes, present a polynomial time algorithm for its solution. If not, show that it is $\mathcal{NP}$ -complete.

Given an undirected acyclic graph $G(V, E)$ and an integer k , decide whether G is k colorable.

18. Is the following problem in $\mathcal{P}$? If yes, present a polynomial time algorithm; if not, show that it is $\mathcal{NP}$ -complete.

Input are an undirected graph $G(V, E)$ and an integer $1 \leq k \leq |V|$. Also, assume that the degree of each node in G is $|V| - O(1)$. The problem is to check whether G has a vertex cover of size k .

19. Assume that there is a polynomial time algorithm CLQ to solve the CLIQUE decision problem.
- (a) Show how to use CLQ to determine the maximum clique size of a given graph in polynomial time.
 - (b) Show how to use CLQ to find a maximum clique in polynomial time.